Declaration of Original Work for SC2006 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	**Course**	**Lab Group**
Toh Kok Soon	CSC	SCS7
Murali Iniya	CSC	SCS7
Prakriti Muralidharan	CSC	SCS7
Murugesan Chandramohan	CSC	SCS7

Important notes:

1. Name must EXACTLY MATCH the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Project Title: CommuteBuddy

Group Members:

Name	**Student ID**
Toh Kok Soon	U2423531K
Murali Iniya	U2421833K
Prakriti Muralidharan	U2422005C
Murugesan Chandramohan	U2420232A

1. Purpose

1.1. Project Mission Statement

To improve the convenience and efficiency of Singapore's transport systems through the use of data that helps commuters make better-informed decisions and urban developers plan transport infrastructures better.

Primary Target Audience:

- Daily Commuters (students, office workers, drivers, etc.)

Secondary Target Audience:

- Urban Planner (agencies that manage passenger load and plan for high-demand periods)
- Carpark Managers (HDB, LTA, URA, etc.)

2. Requirement Elicitation

2.1. Legacy Product Study

2.1.1. Carpark Availability Apps

Primitive carpark availability apps tend to use a list to represent carparks instead of doing it on a map, which makes it hard for users to get an overview.

2.2. Competitive Product Study

2.2.1. carparksg.com

Not many car park availability apps are available, especially on the Google Play Store. The most similar product to our CommuteBuddy carpark availability system that we have identified is a web application found on the site carparksg.com (*Real-time carpark availability in Singapore, n.d.*).

Based on our experiences with the application, we have taken note of the following:

Good	Bad

• Map view enables smooth navigation, such as zooming on clusters of parking lots by clicking on the cluster icon.
• Information is abstracted when zoomed out, therefore improving readability and avoiding overwhelming users with dense information.
	• UI is not straight away intuitive:
	  ○ More lots are red and fewer lots are blue.
	  ○ Red markers for carpark with lots and grey markers for carpark with no lots remaining.
	• The numbers shown show how many

• The app works on both PC and mobile.	carparks there are in the vicinity, but do not show lots availability. • No way to add carparks outside of their limited data. • Users can favourite carparks, but the data is not tied to a user account.

2.2.2. Google Maps

Most of the core functionality that are planned for our public transport planning system have already been covered by Google Maps, what separates us from Google Maps is our focus on a transport app which also includes carpark availability therefore we are able to cater our UI to be more friendly towards our target audience. Furthermore, it would also be a more convenient app for commuters that both use public transport and drives.

2.3. Functional Requirements

1. The system shall allow users to manage their accounts.
    1.1. The system shall allow users to register an account using email and password.
    1.2. The system shall allow users to log in securely with registered credentials.
    1.3. The system shall store user data in a database to ensure synchronization across devices.
    1.4. The system shall allow user to change their account detail
    1.5. The system shall allow user to change their password when they forget password
    1.6. The system shall use hash to store password for security reasons
2. The system shall allow users to search for and manage carparks.
    2.1. The system shall allow users to view real-time carpark availability on a map.
        2.1.1. The map shall display the number of available lots using a marker
        2.1.2. Map marker should include the number of lots available
        2.1.3. Map marker should be color coded to show percentage of lots available
            2.1.3.1. Red for 25% lots left
            2.1.3.2. Yellow for 50% lots left
            2.1.3.3. Green for 75% lots left
        2.1.4. The map marker should not overlap each other, lots in the vicinity should be grouped into a single marker when zoomed out
    2.2. The system shall allow users to view detailed information about individual carparks whenever data is available
        2.2.1. Detailed information can be viewed by clicking on the map marker of the individual carpark
        2.2.2. Detailed information include gantry heights
        2.2.3. Detailed information include parking rates
    2.3. The system shall allow users to bookmark frequently accessed carparks.
    2.4. The system shall allow users to filter carparks shown on the map by:
        2.4.1. Favourited status

2.4.2. Gantry height
2.4.3. Distance radius
2.5. The system shall allow carpark managers to publish carpark information (e.g., location, lot availability, gantry height, pricing) through Carpark APIs.
2.6. The system shall allow administrators to approve or reject Carpark API key requests from carpark managers.
2.7. The system shall allow user-added carparks to be tested using simulated availability data.
3. The system shall provide public transport planning features.
3.1. The system shall allow users to search and check real-time arrival times for buses and MRT trains.
3.2. The system shall provide peak-hour alerts or disruption alerts to users using historical ridership and real-time crowd data.
3.3. The system shall predict high-demand periods (e.g., national day, new year, large events) using historical data and issue alerts.
3.4. The system can provide alternative route suggestions by selecting their preferred route, or when disruptions occur
3.5. The system allows users to bookmark their selected routes in the user's account. The users can also delete and edit their bookmarks.
3.6. The system allows accessibility users to select an "accessible route" option (e.g., ramps, lifts, etc.)
4. The system shall provide insights for urban planners.
4.1. The system shall allow urban developers to view carpark utilization statistics to identify areas requiring additional parking spaces.
5. The system shall provide meaningful error feedback.
5.1. There should be a designated area to display error messages on forms
5.2. Error messages should appear as pop-ups, wherever no designated area is assigned for the page

2.4. Non-Functional Requirements

<table>
<tr><td rowspan="2">Usability</td><td>New User Learnability and Simplicity
The user interface shall be simple and intuitive.
At least 80% of first-time users shall be able to complete a simple task (e.g., finding a nearby car park) within 2 minutes of first using the app.</td></tr>
<tr><td>Readable Interface
Information displayed shall not be overly dense, and text or numbers shall not</td></tr>
</table>

	overlap.
	Platform Responsive ● Users must be able to see 100% of the content regardless of what platform they are using. ● No content shall be clipped or hidden away due to mobile size issues.
Performance	Data Retrieval Efficiency ● The system shall retrieve and display real-time bus and MRT arrival data within 3 seconds. ● The system shall update carpark availability data at least every 60 seconds
Reliability	Quick App Startup ● In the event of a system crash or reboot, the system shall restore full functionality within 5 minutes.
Supportability	Accessibility ● The system shall be cross-platform and accessible on: ○ Mobile (iOS, Android) ○ Tablets ○ PCs via web interface

2.5. Data Dictionary

2.5.1. Non-Database Data Dictionary

Term	Synonyms	Definition
Application	CommuteBuddy WebApp App Software	The software solution proposed by us
Users		Individuals who interact with the application, including commuters, admins, carpark managers, and urban developers.
User Account		User details needed for authentication, and their credentials

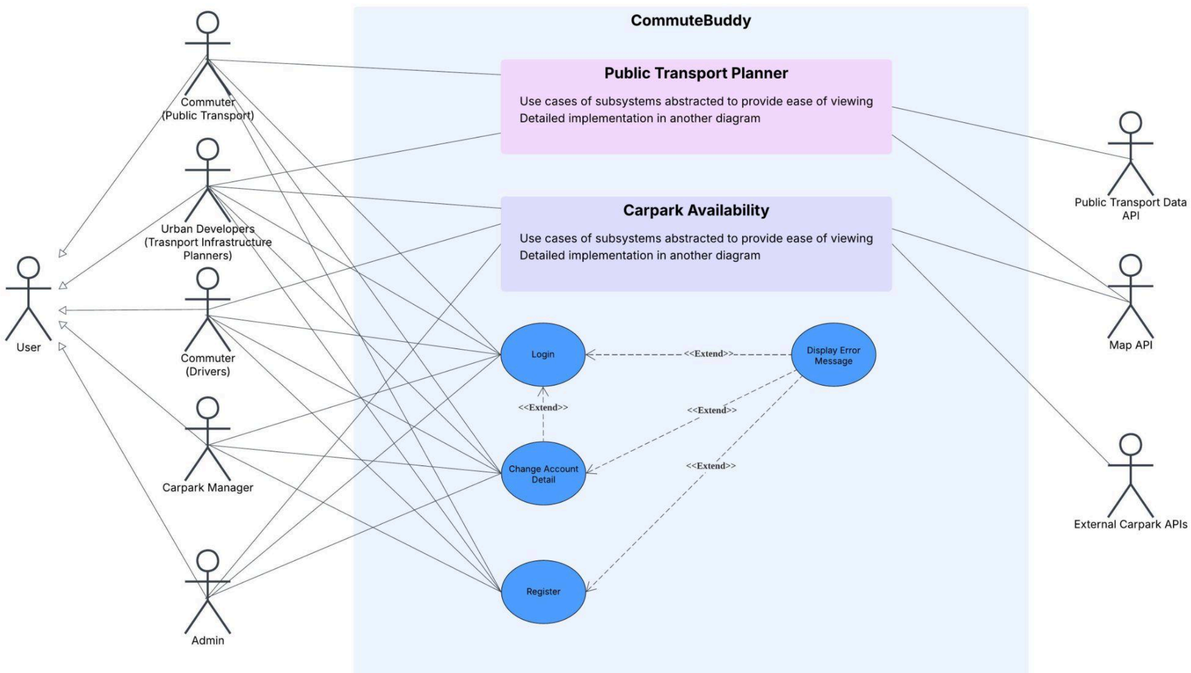
User Role	User Type	Classification of User (e.g., commuters, urban planners, carpark managers) tied to a user account
User Profile		A collection containing user preferences (e.g., preferences, saved addresses).
Commuters	Travellers	The general public that uses transport infrastructures
Admins		Authorized staff who oversee and maintain the application operation
Carpark Manager		Person, Organisation, or Agencies in charge of one or more carparks
Urban Developers	Infrastructure Planners City Planners	Person, Organisation, or Agencies in charge of the allocation of resources for public infrastructure, specifically for transport in this case
Carpark	Parking	A designated parking facility for vehicles.
Carpark Availability		The current number of parking lots available at a car park. (i.e., high availability means more free lots)
Carpark Utilisation		Percentage of carpark lots being used up (i.e. high utilisation means fewer lots available)
Carpark Rate		The cost of parking at a car park is based on time and other factors, such as the day of the week
Gantry Height		The maximum vehicle height allowed at the entrance of a car park.
Carpark Preference Filter		A user-defined criterion for filtering carparks (e.g., cheapest rate, closest distance, gantry height suitable for the vehicle).
Parking Demand		The level of need for parking spaces in an area
Bookmark	Favourite	A feature allowing the saving of an in-app data point
Bus Service		A scheduled bus line, identified by its service number (e.g., 7, 106, 961).
Bus Stop		A designated location where passengers can board or alight from buses.
MRT Line	Train Line	A railway line in Singapore's MRT network (e.g., East-West Line, Circle Line) can also be identified by a 2-letter acronym (e.g., CC, NS, EW)

MRT Station	Train Station	A designated train stop where commuters can board or alight, represented by a 2-letter acronym followed by a number (e.g. NS1, EW21), a train station may include one or more such representations
Bus Interchange		A central transport hub connecting multiple bus services, often connected to a train station
Train Interchange		A train station connecting more than one train line
Route		A path from a starting point to a destination, combining multiple transport modes.
Mode of Transport		The type of travel chosen by the commuter (e.g., MRT, bus, walking, private car, multi-modal).
Estimated Travel Time (ETT)	Estimated Time of Arrival (ETA)	The predicted duration of a journey along a given route, considering traffic and delays.
Peak Hour		An hourly period where travel demand is higher than usual
Peak Hour Alert	Peak Hour Notification	A notification for peak hour
Crowd Heatmap		A visual representation showing the density of commuters in specific locations(stations, bus interchanges, routes) by colour.
Home Address		The commuter’s saved starting point, often the default for route planning.
Saved Route		A route bookmarked by the user for easy reuse.
Recent Trip		The most recently completed or planned journey by a user, showing origin, destination, mode of transport, and travel time.
Travel Demand		The volume of commuters expected at a certain place and time.
High-Demand Period		A recurring or event-driven time when passenger load is significantly above average (e.g., morning peak, National Day).

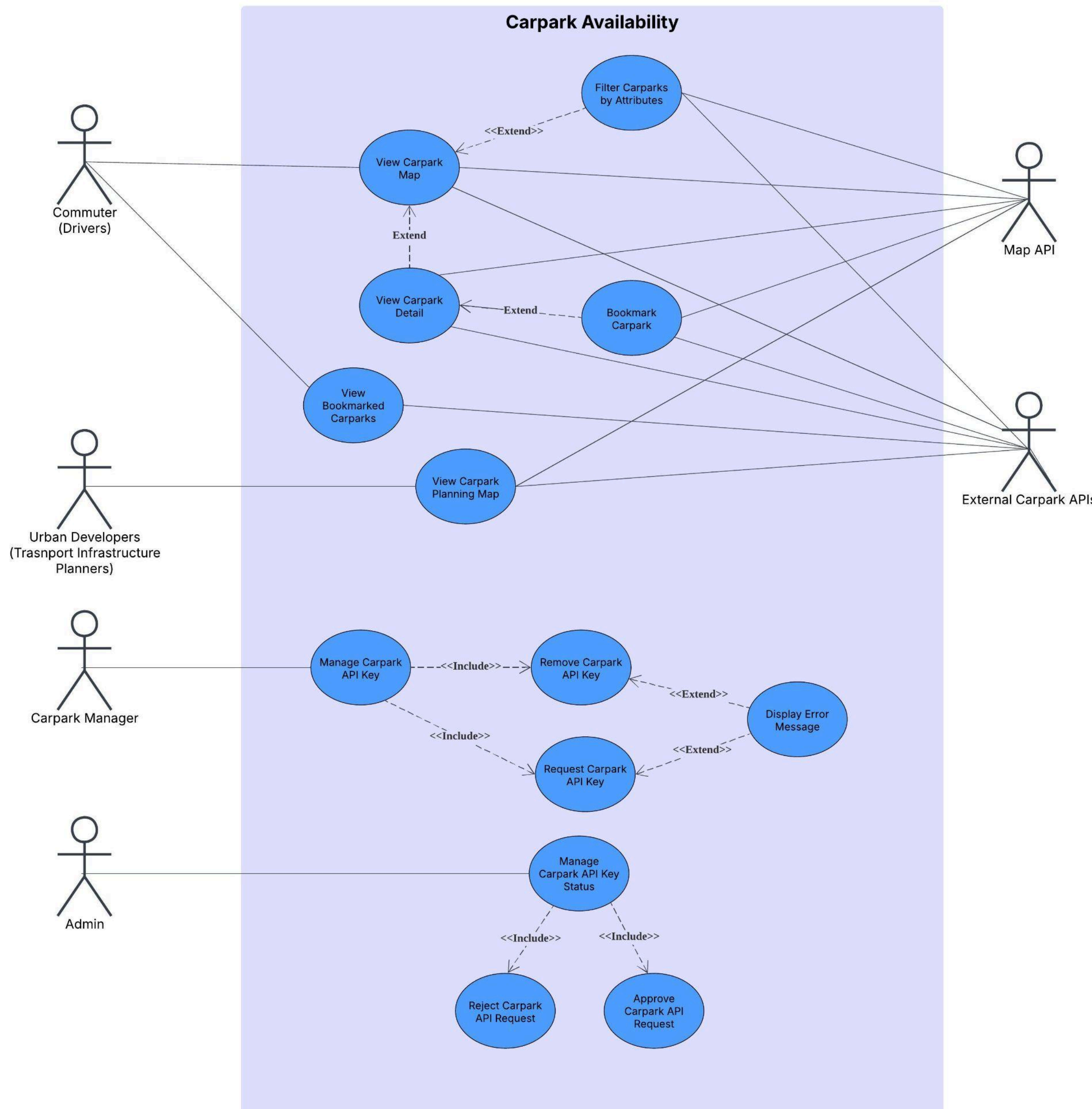
3. Prototype

3.1. Use Case Diagrams

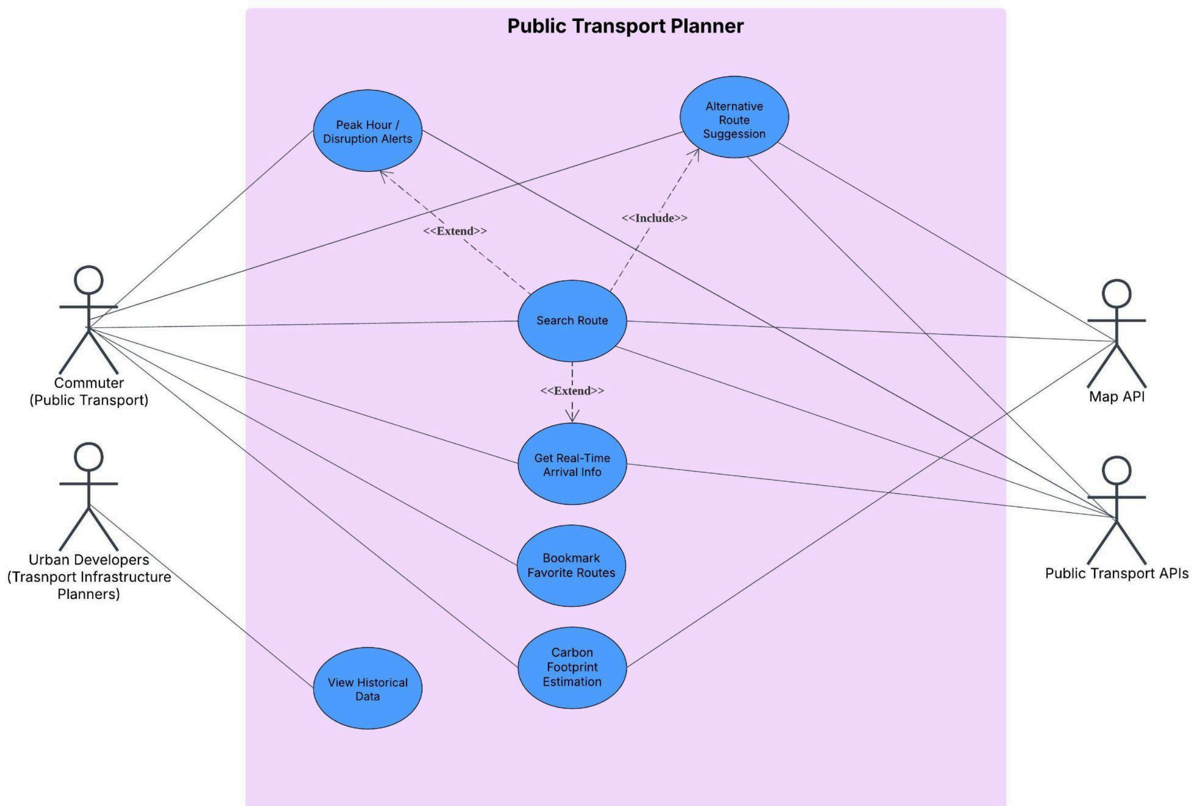
3.1.1. General Operations



3.1.2. Carpark Availability System



3.1.3. Public Transport Planner System



3.2. Use Case Descriptions

3.2.1. Functional Requirement #1: Account Management

3.3.1.1 Login

Use Case ID:	UC1		
Use Case Name:	Login		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		
Description:	Allows the app to identify the user from the details they entered		
Preconditions:	• User has an account		

Postconditions:	• Application saves the logged-in state of the user
Priority:	High
Frequency of Use:	Medium to High
Flow of Events:	1. User enters email and password, and then presses login 2. The format of the email is checked 3. The entered detail is passed to the application backend for authentication 4. Look for the username in the database and check its corresponding password hash for a match 5. Sends the user back to the page they were on
Alternative Flows:	<u>AF-S1:</u> 1. If user click “forgot password”, goes to use case 3 (Change Account Detail) <u>AF-S2:</u> 1. If the format of the email is wrong, the error message “Invalid Email Format” is displayed on the login screen <u>AF-S4:</u> 1. If an account with the email is not found or the password does not match 2. The error message “Invalid login information" is displayed on the login screen
Exceptions:	EX1: email format is wrong → AF-S2 EX2: user account not found → AF-S4
Includes:	-
Special Requirements:	-
Assumptions:	• Database connection available
Notes and Issues:	The invalid login information message is deliberately kept vague for security reasons

3.3.1.2 Register

Use Case ID:	UC2		
Use Case Name:	Register		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User (except Admin)		

Description:	Allows the user to create an account
Preconditions:	● User has no account
Postconditions:	● Account is created and saved in the database
Priority:	Medium
Frequency of Use:	Low
Flow of Events:	1. User fill up registration form: a. User enter their nickname b. User re-type email c. User enters password d. User re-type password e. User press register 2. Input is validated to ensure correctness 3. User account is created in the database
Alternative Flows:	AF-S2: 1. If input validation fails, collect all errors 2. Display error messages on screen and wait for user to click ok 3. Goes back to step 1 of main flow
Exceptions:	EX1: invalid nickname format → AF-S2 EX2: invalid email format → AF-S2 EX3: email already in used → AF-S2 EX4: email does not match → AF-S2 EX5: password does not meet the requirement → AF-S2 EX6: password does not match → AF-S2
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

3.3.1.3 Change Account Detail

Use Case ID:	UC3		
Use Case Name:	Change Account Detail		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		

Description:	Allows the user to change their account credentials
Preconditions:	• User has an account
Postconditions:	• Account details changed are saved in the database
Priority:	Medium
Frequency of Use:	Low
Flow of Events:	1. User goes to the profile page 2. They can enter a new email or a new password and click update 3. The details are saved in the database
Alternative Flows:	AF-S1: 1. If user is not logged in, they can change their password by clicking “forgot password” from login page 2. User have to enter their email 3. A random password will be generated and saved into the database for the account with the email 4. An email with the randomly generated password will be sent to the user email AF-S2: 1. If the email they entered is already in use, display error message “email already in use” 2. skip step 3
Exceptions:	EX1: invalid email format → user have to re-enter their email EX2: user account with the email address is not found → display error message “user account not found” EX3: email already in use → AF-S2
Includes:	-
Special Requirements:	-
Assumptions:	• Database connection is available
Notes and Issues:	-

3.2.2. Functional Requirement #2: Searching and Managing of Carparks

3.3.2.1 Manage Carpark API Key

Use Case ID:	UC4
Use Case Name:	Manage Carpark API Key

Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Carpark Managers		
Description:	A collection of interactions involving Carpark API Keys		
Preconditions:	• User is logged in		
Postconditions:	-		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. Carpark Managers can select: a. Request API Key b. Remove API Key		
Alternative Flows:	-		
Exceptions:	-		
Includes:	• Request API Key (UC5) • Remove API Key (UC6)		
Special Requirements:	-		
Assumptions:	-		
Notes and Issues:	-		

3.3.2.2 Request Carpark API Key

Use Case ID:	UC5		
Use Case Name:	Request Carpark API Key		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Carpark Manager		
Description:	Allow user to ask for a carpark API key		
Preconditions:	• User is logged in		
Postconditions:	• API key request is saved in the database		

Priority:	Medium
Frequency of Use:	Low
Flow of Events:	1. User goes to settings 2. User clicks “manage my api keys” 3. User clicks on “request API key” 4. A form will appear for user: a. User inputs an API endpoint 5. Generate an API key 6. Create a database entry with the API status as pending
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

3.3.2.3 Remove Carpark API Key

Use Case ID:	UC6		
Use Case Name:	Remove Carpark API Key		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Carpark Manager		
Description:	Allows user to delete an existing carpark API key		
Preconditions:	● User is logged in ● User has carpark API key		
Postconditions:	● API key is removed from the database		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. User goes to “settings” 2. User clicks on “manage my api keys”		

	3. User clicks “remove api key” on the key that they want to remove 4. Look for the API Key in the database 5. Remove API Key from database
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

3.3.2.4 Manage Carpark API Key Status

Use Case ID:	UC7		
Use Case Name:	Manage Carpark API Key Status		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Admin		
Description:	A collection of actions used by the admin involving API Keys status		
Preconditions:	● User is logged in ● User is admin		
Postconditions:	-		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. Admin can select to approve or reject API key from a list		
Alternative Flows:	-		
Exceptions:	-		
Includes:	● Reject API Request ● Approve API Request		
Special	-		

Requirements:	
Assumptions:	-
Notes and Issues:	-

3.3.2.5 Reject API Key Request

Use Case ID:	UC8		
Use Case Name:	Reject Carpark API Key Status		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Admin		
Description:	Deny the request for an API Key		
Preconditions:	● User is logged in ● User is admin		
Postconditions:	● The API key is deleted from the database		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. Deletes the API Key request entry from the database 2. Sends an email to notify the user		
Alternative Flows:	-		
Exceptions:	-		
Includes:	-		
Special Requirements:	-		
Assumptions:	-		
Notes and Issues:	-		

3.3.2.6 Approve API Key Request

Use Case ID:	UC9

Use Case Name:	Approve Carpark API Key Status		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	Admin		
Description:	Approve the request for an API Key		
Preconditions:	• User is logged in • User is admin		
Postconditions:	• API key status in the database is updated to active		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. Change the status of the API in the database to active 2. Send an email to alert the user		
Alternative Flows:	-		
Exceptions:	-		
Includes:	-		
Special Requirements:	-		
Assumptions:	-		
Notes and Issues:	-		

3.3.2.7 View Carpark Map

Use Case ID:	UC10		
Use Case Name:	View Carpark Map		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Visual representation of carpark availability		
Preconditions:	-		
Postconditions:	-		

Priority:	Medium
Frequency of Use:	High
Flow of Events:	1. Checks if the user has geolocation permission on 2. Get carpark data from the database (see notes for details) 3. Render map with carpark data through Maps API 4. Centers map on user's current location
Alternative Flows:	AF-S1: 1. Ask for the user's geolocation permission AF-S4 1. If unable to access user current location, centers on a default location
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: Unable to access user geolocation → AF-S4 EX3: External Carpark API not responding → Display an error message “No Carpark Data Available” and render an empty map EX4: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	● Map marker should be color coded to reflect percentage of lots available ● Map marker should include number of lots available
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	Carpark data, which includes availability of lots and location coordinates will be updated periodically in the database through API calls to external carpark APIs

3.3.2.8 Filter Carparks by Attributes

Use Case ID:	UC11		
Use Case Name:	Filter Carpark by Attributes		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Allows the user to select the kind of car park that is shown on the map		

Preconditions:	-
Postconditions:	-
Priority:	Medium
Frequency of Use:	High
Flow of Events:	1. The user can select from various dropdown menus, such as gantry height and carpark type 2. The program takes in the user preferences and renders them on the map using the Map API
Alternative Flows:	-
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” and render an empty map EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	-

3.3.2.9 View Carpark Detail

Use Case ID:	UC12		
Use Case Name:	View Carpark Detail		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Show detailed information about the individual carparks		
Preconditions:	-		
Postconditions:	-		

Priority:	Medium
Frequency of Use:	Medium
Flow of Events:	1. The user clicks on the marker representing the car park on the map 2. A small pop-up window will list out all the details of the car park
Alternative Flows:	AF-S1: 1. If the marker the user clicks represents multiple carparks in the vicinity, zoom the map in to display more markers
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” and render an empty map EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● The map marker the user clicked on represent an individual carpark → AF-S1
Notes and Issues:	-

3.3.2.10 Bookmark Carpark

Use Case ID:	UC13		
Use Case Name:	Bookmark Carpark		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Allow the user to save carparks that are frequently accessed		
Preconditions:	User is logged in		
Postconditions:	Bookmarked carparks are saved in the database		

Priority:	Low
Frequency of Use:	Medium
Flow of Events:	1. User accesses the individual carpark through the "view carpark detail" use case 2. User clicks on the bookmark to save the carpark
Alternative Flows:	-
Exceptions:	EX1: Map API not responding → Display an error message "No Map Data available" EX2: External Carpark API not responding → Display an error message "No Carpark Data Available" EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	-

3.3.2.11 View Bookmarked Carparks

Use Case ID:	UC14		
Use Case Name:	View Bookmarked Carparks		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), External Carpark API		
Description:	A list view of all the user car parks		
Preconditions:	User is logged in		
Postconditions:	-		
Priority:	Low		
Frequency of Use:	Medium		

Flow of Events:	1. User enters “bookmarked carparks” 2. The program gathers bookmarked car parks from the database 3. The program then populates the bookmarked carpark with its detailed information gathered from the External Carpark API into a list view
Alternative Flows:	-
Exceptions:	EX1: No bookmarked carparks → Show empty list EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Carpark API connections are available
Notes and Issues:	-

3.3.2.12 View Carpark Planning Map

Use Case ID:	UC15		
Use Case Name:	View Carpark Map		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Urban Developers, Map API, External Carpark API		
Description:	Visual representation of carpark availability		
Preconditions:	-		
Postconditions:	-		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. Using data from the Map API and External Carpark API, the program generates a map that shows regions where carpark		

	utilisation is high, suggesting a need for more carparks 2. Users who are urban developers may access this map to gain insight into where carpark infrastructure is needed
Alternative Flows:	-
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	-

3.2.3. Functional Requirement #3: Public Transport Planning Features

3.3.3.1 Peak Hour/ Disruption Alerts

Use Case ID:	UC16		
Use Case Name:	Peak Hour/ Disruption Alerts		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (Public transport), Public Transport Data APIs		
Description:	The system provides commuters with alerts about peak hours or service disruptions, enabling them to plan their trips efficiently.		
Preconditions:	● User is logged in to the system.		
Postconditions:	● Users are notified of disruptions/ peak hour congestion via app alerts.		
Priority:	High		
Frequency of Use:	High (daily, especially during commuting hours)		

Flow of Events:	1. The user searches or selects a route by entering the starting point and the destination 2. System queries Public Transport Data APIs for live data. 3. System checks for congestion/disruption alerts. 4. Alerts are displayed to the user.
Alternative Flows:	AF-S1: 1. If no disruptions exist, only the peak hour advisory is shown
Exceptions:	EX1: API downtime → no alerts available.
Includes:	Search Route (UC17)
Special Requirements:	-
Assumptions:	● Public transport APIs provide up-to-date alerts.
Notes and Issues:	-

3.3.3.2 Search Route

Use Case ID:	UC17		
Use Case Name:	Search Route		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (Public transport), Maps API		
Description:	The user searches for the optimal public transport route between two points.		
Preconditions:	● User has the app open and is ready to perform a search.		
Postconditions:	● A list of possible routes is displayed.		
Priority:	Critical (core function)		
Frequency of Use:	Very frequent (for each trip planned)		
Flow of Events:	1. User enters start and end locations 2. System queries the Map API 3. Uses the Map API to draw the route 4. Routes are displayed with travel times		
Alternative Flows:	AF-S1:		

	1. User can refine search with filters (e.g., mode of transport)
Exceptions:	EX1: Invalid input (invalid location input) → user has to re-enter these locations
Includes:	Alternative Route suggestions (UC18)
Special Requirements:	Must integrate with the live traffic data
Assumptions:	• An internet connection must be available
Notes and Issues:	Ensure that there is a reasonable query response time

3.3.3.3 Alternative Route Suggestion

Use Case ID:	UC18		
Use Case Name:	Alternative Route Suggestion		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (Public transport)		
Description:	Provides alternative travel routes in case of disruptions or user preference.		
Preconditions:	• The user must have initiated a route search based on location		
Postconditions:	• Alternative routes are also displayed		
Priority:	Medium		
Frequency of Use:	Medium (depends on whether disruptions or personal preferences arise)		
Flow of Events:	1. User searches for a route (UC17) 2. The system generates additional suggestions 3. User selects preferred route		
Alternative Flows:	<u>AF-S1:</u> 1. If there is only 1 viable route that exists, the system displays that route only		
Exceptions:	EX1: The API does not return the alternative routes the user did not choose		

Includes:	Search Route (UC17)
Special Requirements:	Must consider time, distance, cost, personal preferences, etc
Assumptions:	• Maps API can support alternative routes
Notes and Issues:	May need user preferences (e.g., fewer transfers, etc.)

3.3.3.4 Get real-time arrival information

Use Case ID:	UC19		
Use Case Name:	Get real-time arrival information		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (Public transport), Public Transport Data APIs		
Description:	Displays live bus/train arrival timings to users		
Preconditions:	• Route is selected, API connection is available		
Postconditions:	• Live arrival times are shown		
Priority:	High		
Frequency of Use:	High (for every trip)		
Flow of Events:	1. User selects a route 2. System queries Public Transport APIs 3. Arrival info is displayed on the app		
Alternative Flows:	AF-S1: 1. If there is data displayed, show the approximate schedules		
Exceptions:	EX1: API is unavailable → fallback to timetable data		
Includes:	Search route (UC17)		
Special Requirements:	Refresh interval every 60 seconds		
Assumptions:	• Real-time API feeds need to be reliable		
Notes and Issues:	Accuracy → gains the user's trust		

3.3.3.5 Bookmark favourite routes

Use Case ID:	UC20		
Use Case Name:	Bookmark favourite routes		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (public transport)		
Description:	Allows the user to save frequently used routes		
Preconditions:	● User is logged in		
Postconditions:	● Routes are stored under the user's profile		
Priority:	Medium		
Frequency of Use:	Frequent (daily commuters)		
Flow of Events:	1. User searches/ selects a route 2. User bookmarks it 3. Route is saved for future use/ quick access		
Alternative Flows:	AF-S2: 1. If the route is already bookmarked, unbookmark it 2. Skip step 3		
Exceptions:	EX1: Storage/ database error		
Includes:	-		
Special Requirements:	Cloud sync with the account		
Assumptions:	● User accounts support these features		
Notes and Issues:	Manage storage for multiple bookmarks		

3.3.3.6 Manage user preferences

Use Case ID:	UC21
Use Case Name:	Manage user preferences

Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (public transport)		
Description:	Users can set travel preferences (e.g., mode of transport).		
Preconditions:	● User is logged in		
Postconditions:	● Preferences stored and used in route planning		
Priority:	Medium		
Frequency of Use:	Occasional (when updating the settings)		
Flow of Events:	1. User opens preferences 2. User selects preferred travel options 3. System saves those preferences		
Alternative Flows:	AS-F1: 1. User resets preferences to default (if they want to)		
Exceptions:	EX1: Preferences are not saved due to a system error		
Includes:	-		
Special Requirements:	Preferences should persist across all devices		
Assumptions:	-		
Notes and Issues:	Personalisation enhances the adoption of our app		

3.2.4. Functional Requirement #4: Urban Planning Insights

3.3.4.1 View Historical data

Use Case ID:	UC22		
Use Case Name:	View historical data		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Urban developers (Transport Infrastructure Planners)		
Description:	Provides past transport usage data to planners		

Preconditions:	● The system stores sufficient history
Postconditions:	● Data visualised/ reported to developers
Priority:	Medium
Frequency of Use:	Low to Medium (periodic analysis)
Flow of Events:	1. The developer requests data 2. System retrieves historical transport data 3. Data is displayed/exported
Alternative Flows:	-
Exceptions:	EX1: Data is incomplete/ missing
Includes:	-
Special Requirements:	Secure access to data
Assumptions:	● Data is stored reliably
Notes and Issues:	-

3.2.5. Functional Requirement #5: Display of Error Messages

3.3.5.1 Display Error Message

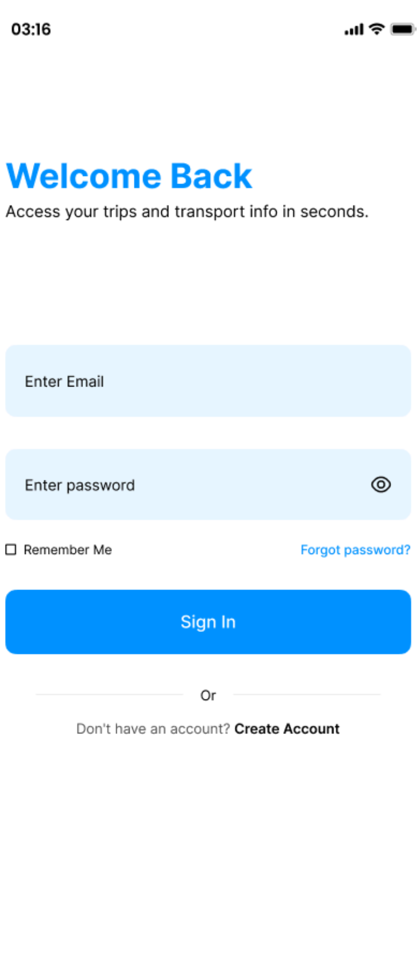
Use Case ID:	UC23		
Use Case Name:	Display Error Message		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		
Description:	Show error text		
Preconditions:	● User has no account		
Postconditions:	● Account is created and saved in the database		
Priority:	Medium		
Frequency of Use:	Low		
Flow of Events:	1. Display a Pop-up with an error message		

Alternative Flows:	AF-S1: 1. If a designated area for an error message is found, the error message will be rendered on the page itself, and no pop-up will be shown
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	An example of a designated area for an error message can be at the bottom of forms, such as login and registration

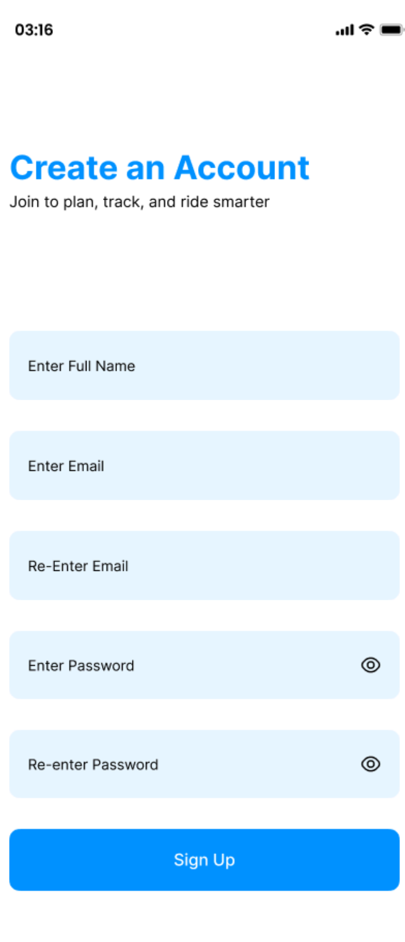
3.2.6. UI Mock Ups

Account Management

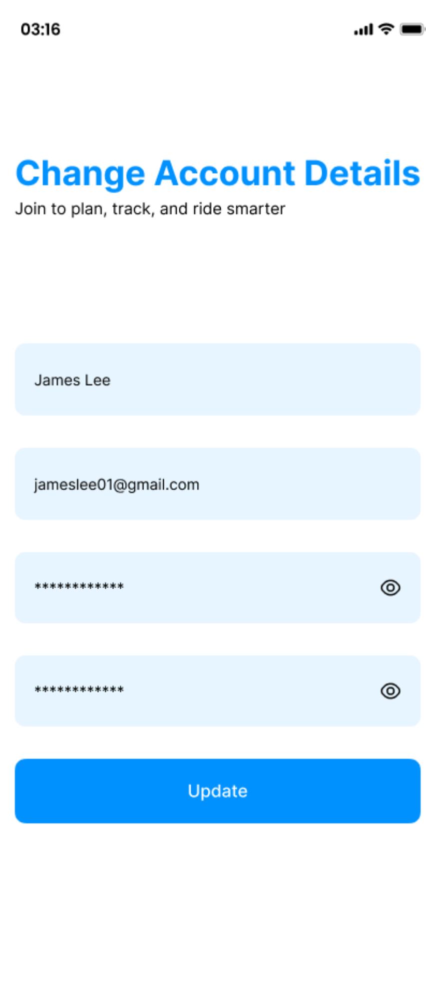
Login Page (UC1)



Registration Page (UC2)

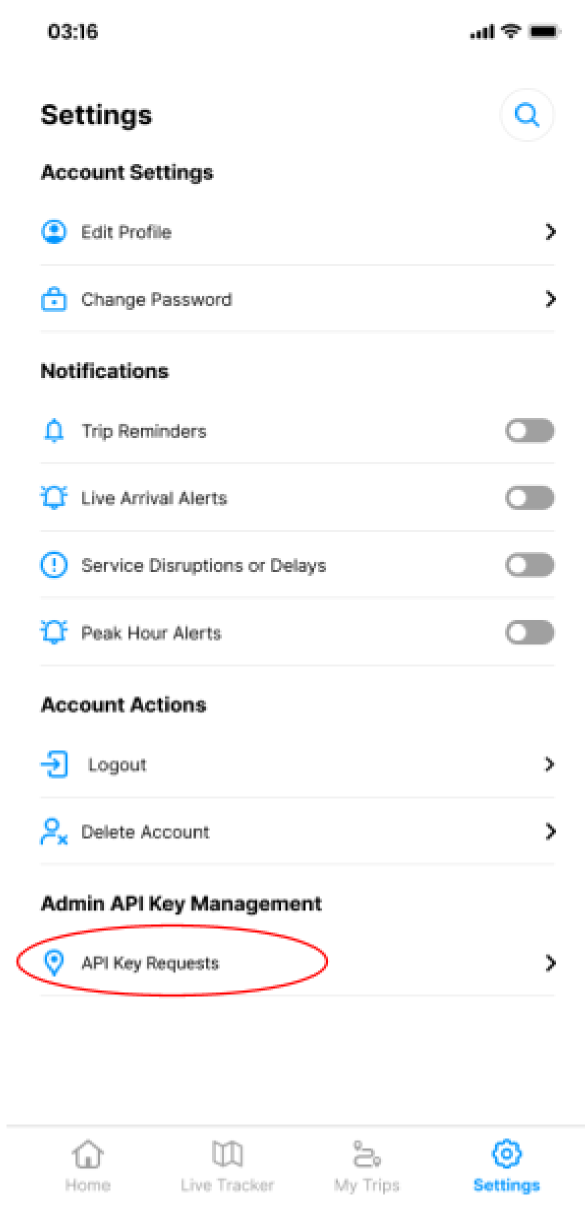


Change Account Details (UC3)

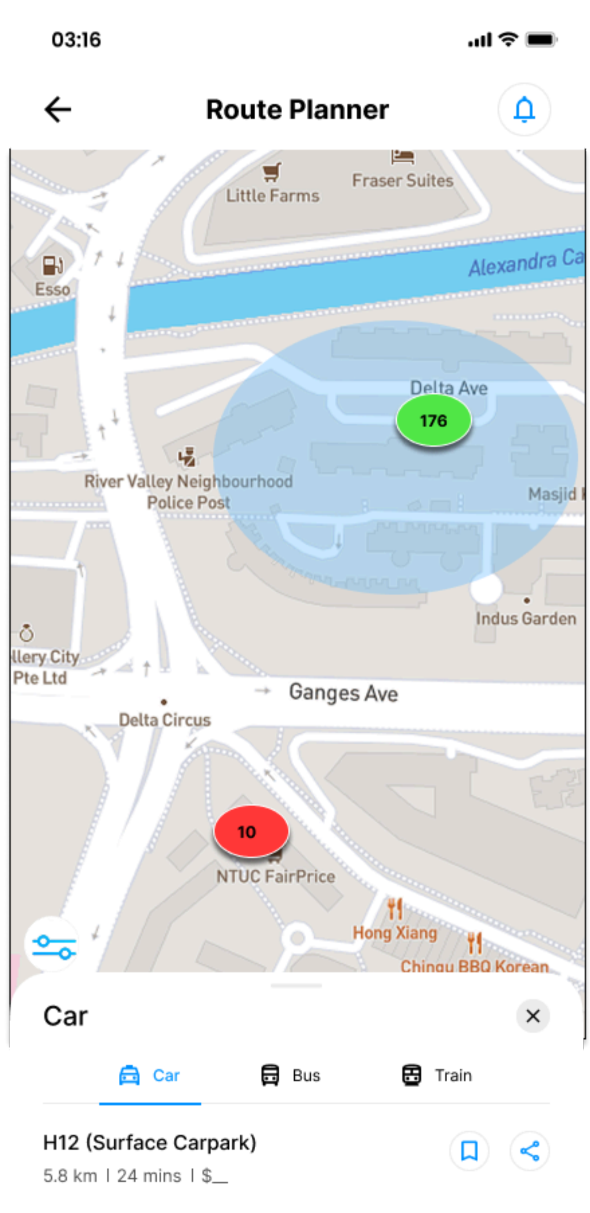


Carpark Availability

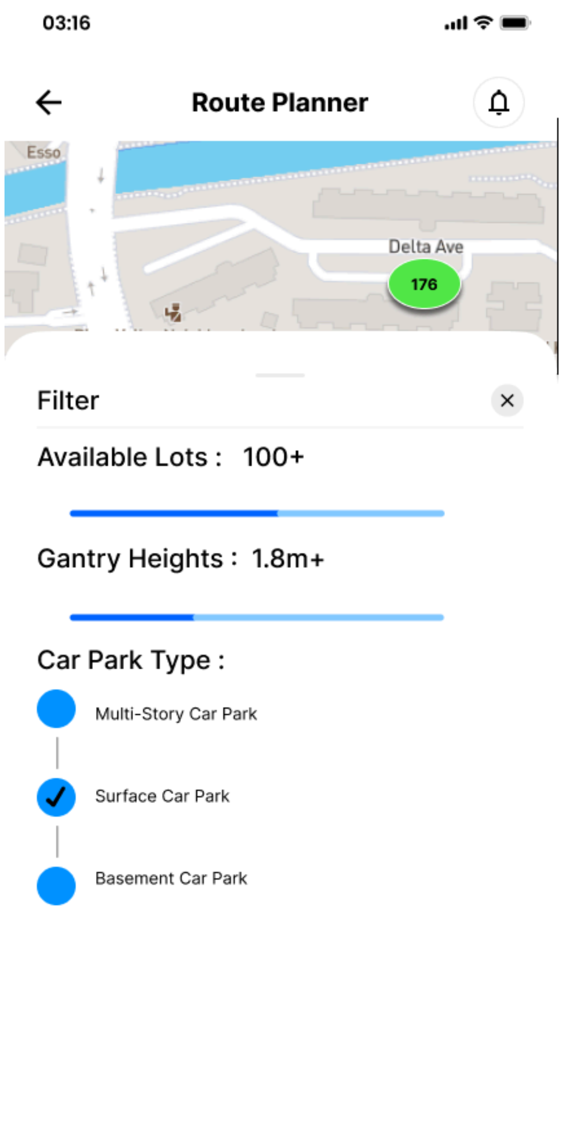
Admin Setting Page - For approval and rejection of API Key (UC7)



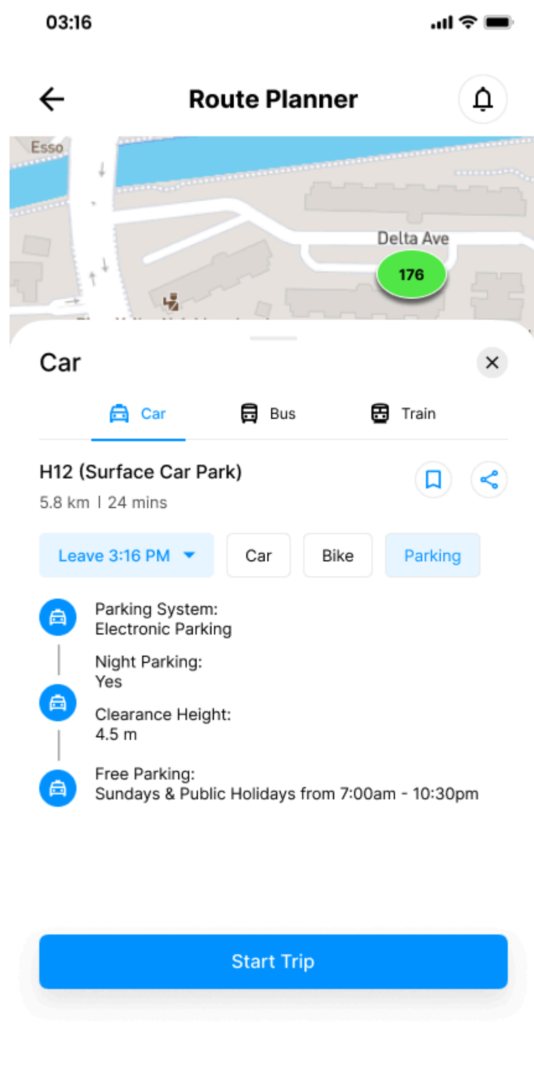
Carpark Map (UC10)



Carpark Filters (UC11)

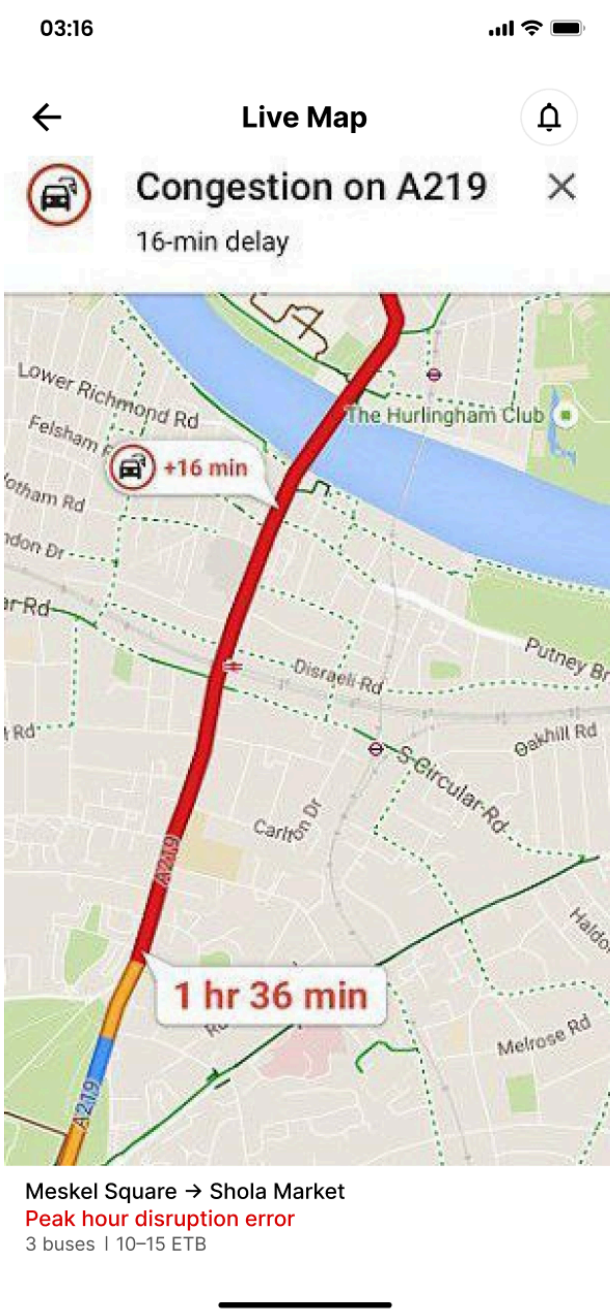


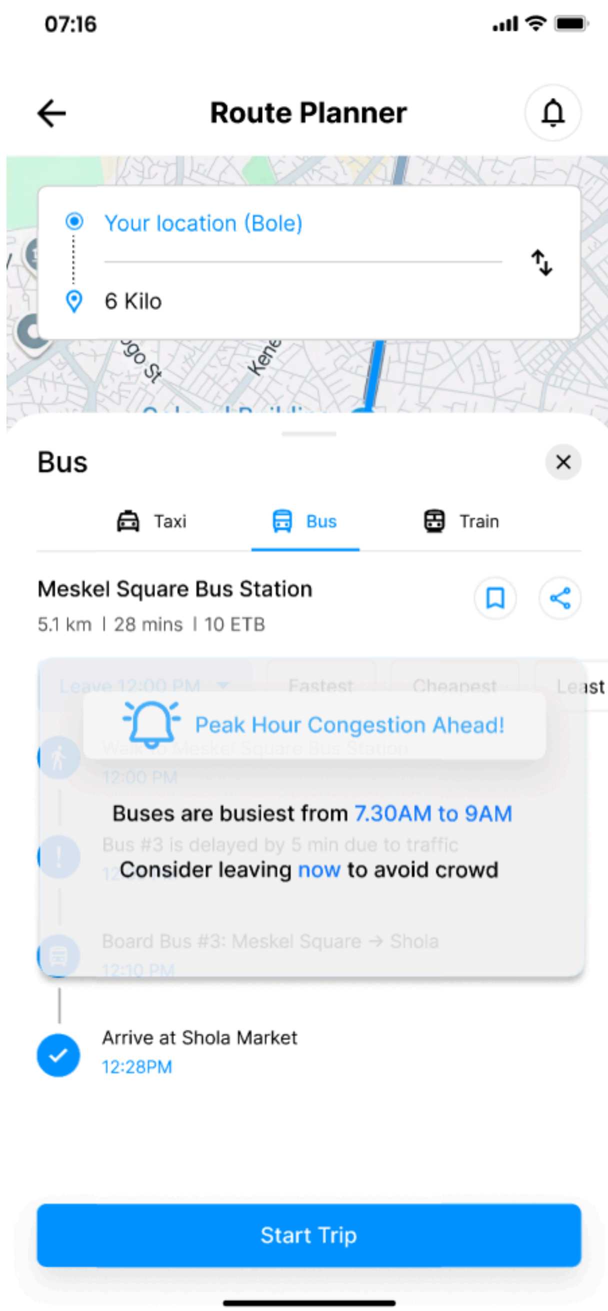
Carpark Details (UC12)



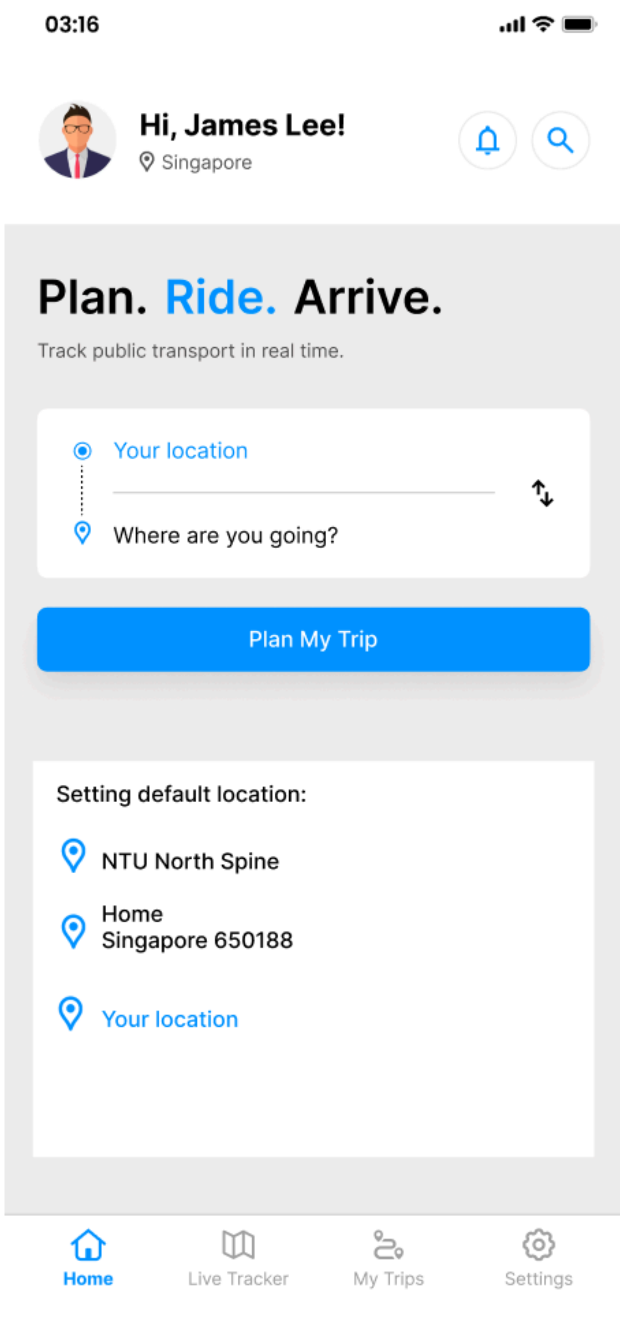
Public Transport Planner

Peak hour/ Disruption Alerts (UC16):

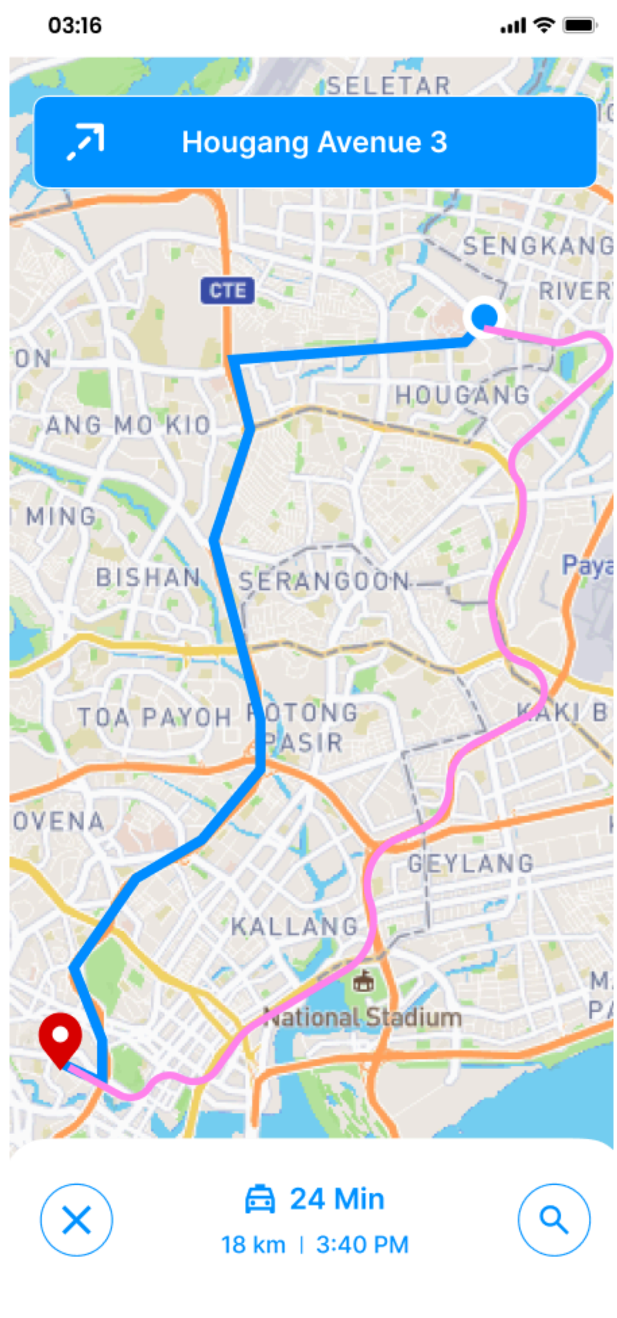




Search Route (UC17):

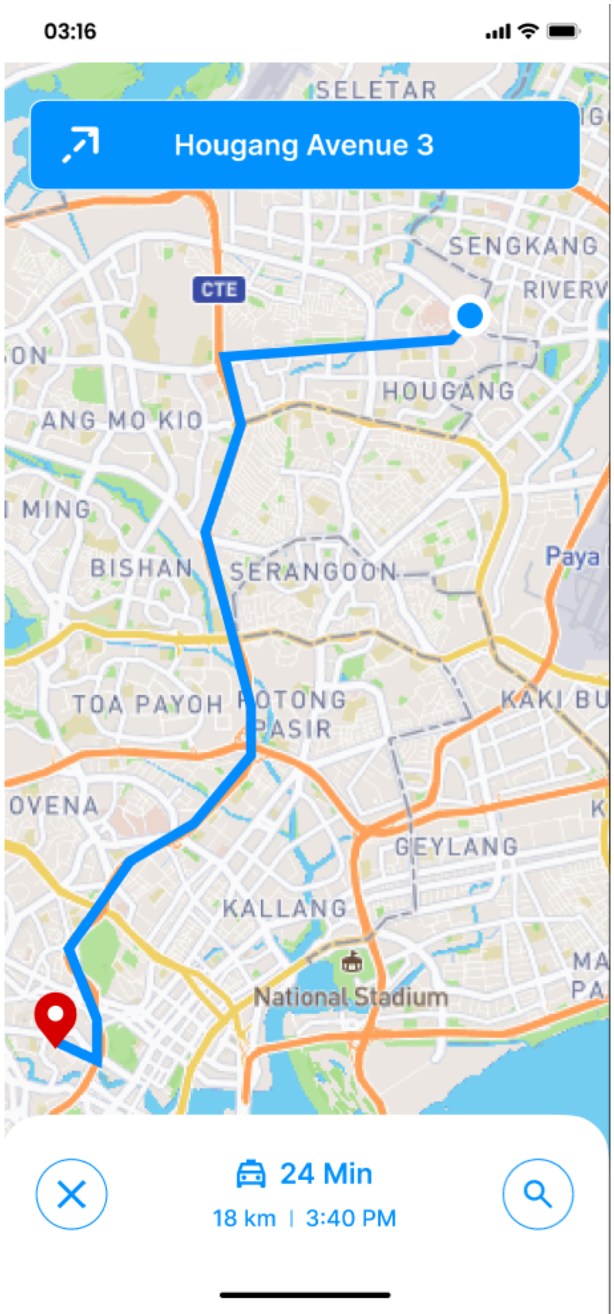


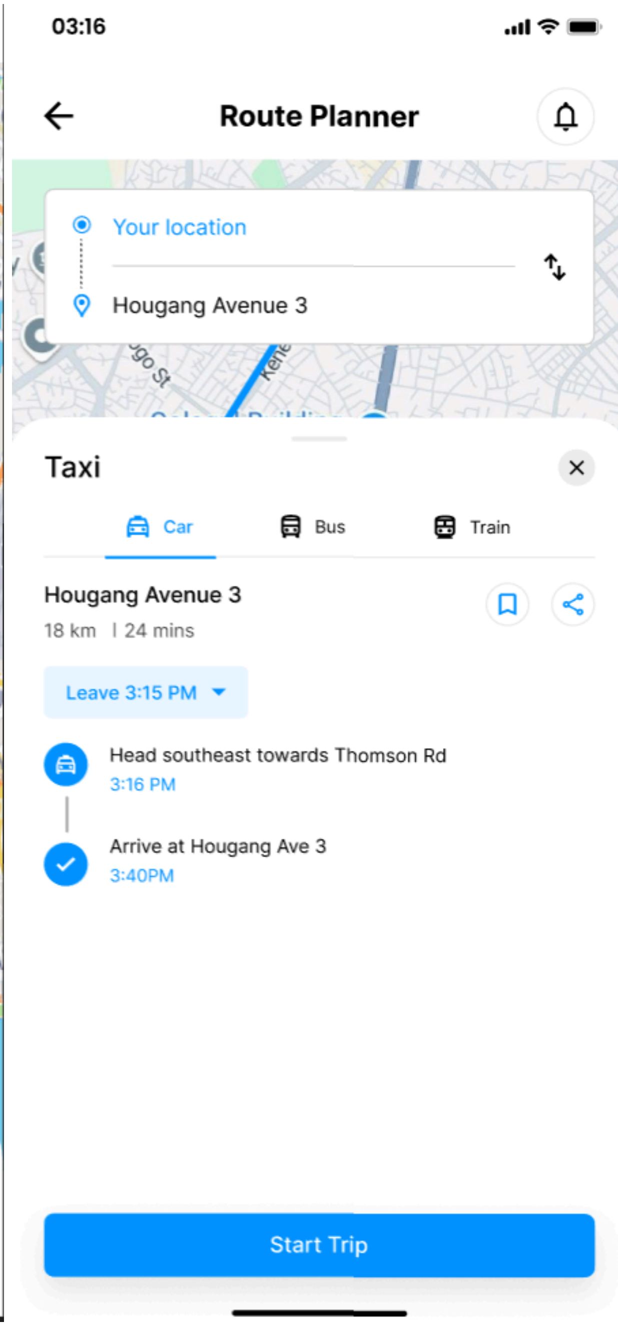
Alternative Route Suggestion (UC18):



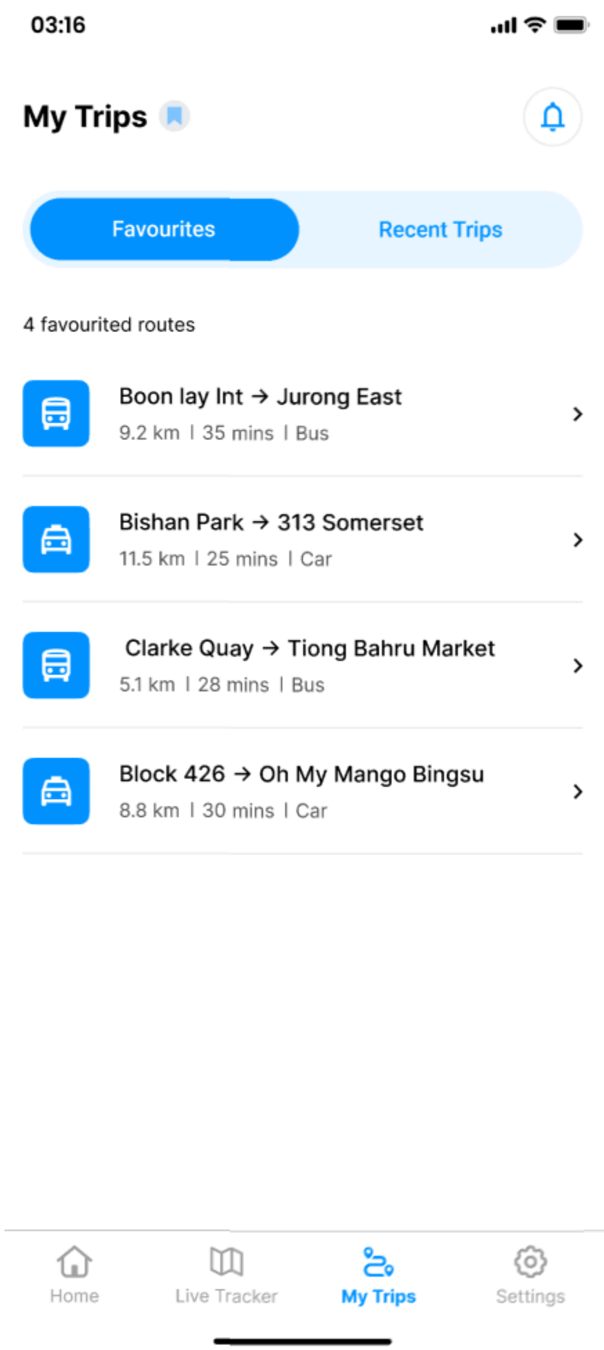
Blue - Suggested route
Pink- Alternative route

Get real-time arrival information (UC19):

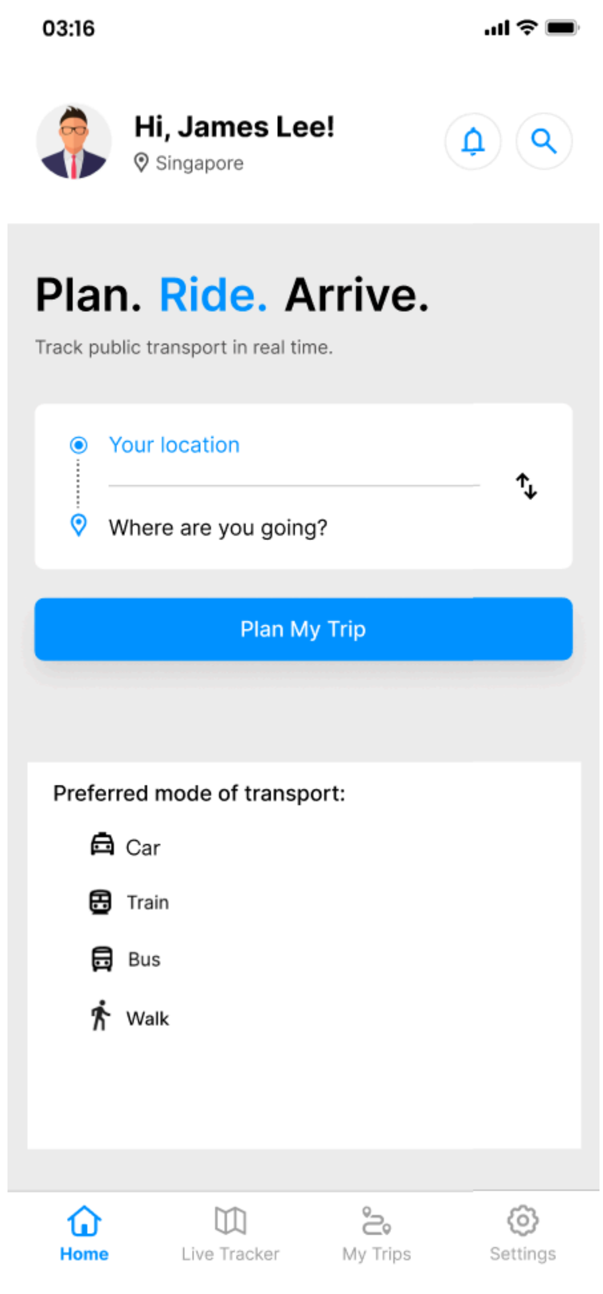




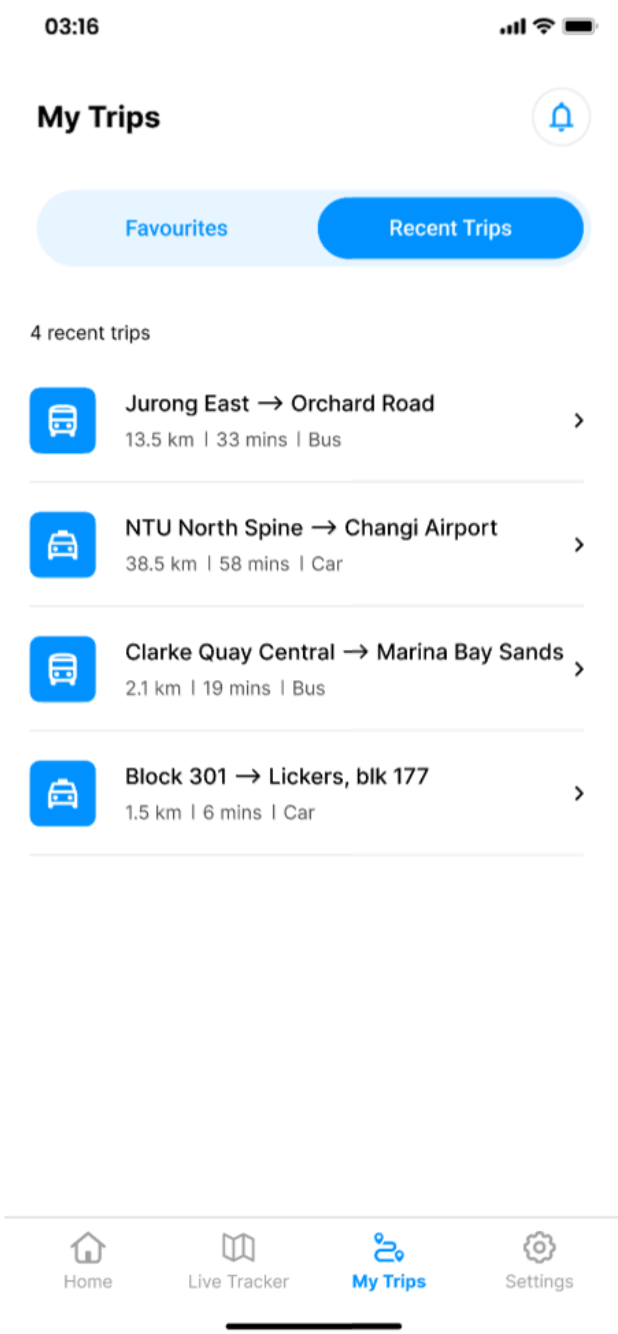
Bookmark favourite routes (UC20):



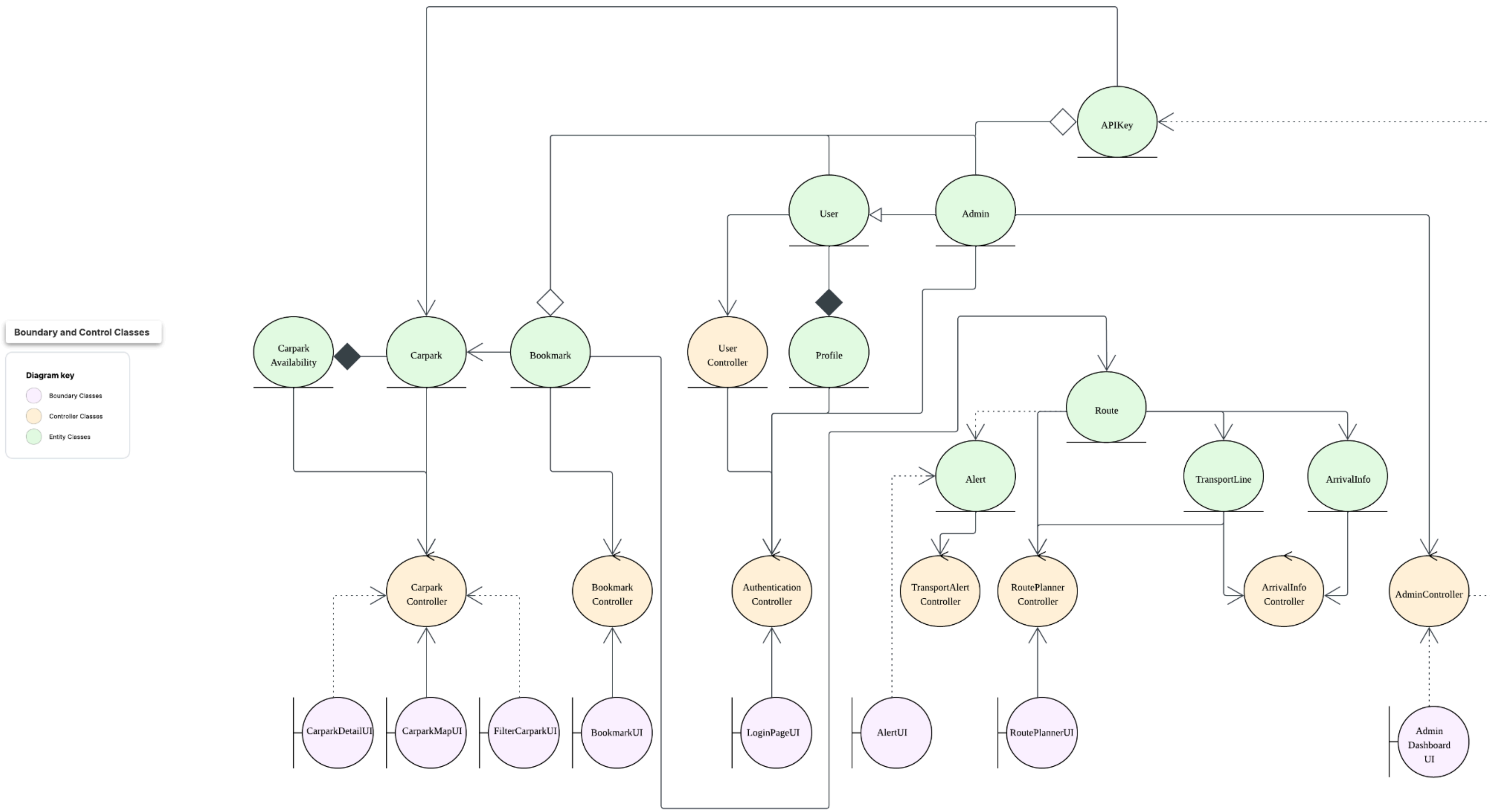
Manage user preferences (UC21):



View historical data (UC22):

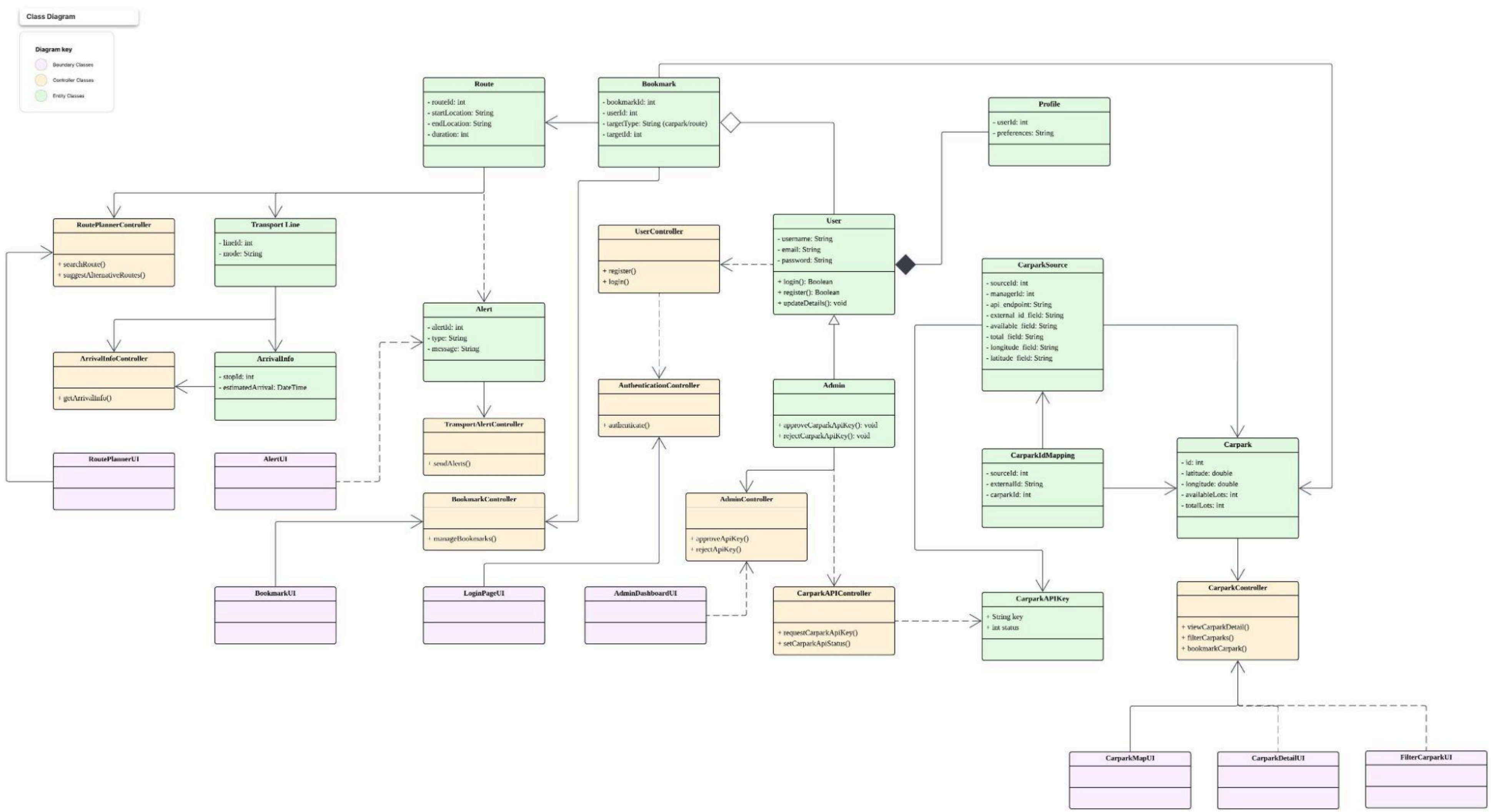


4. Key boundary classes and control classes



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

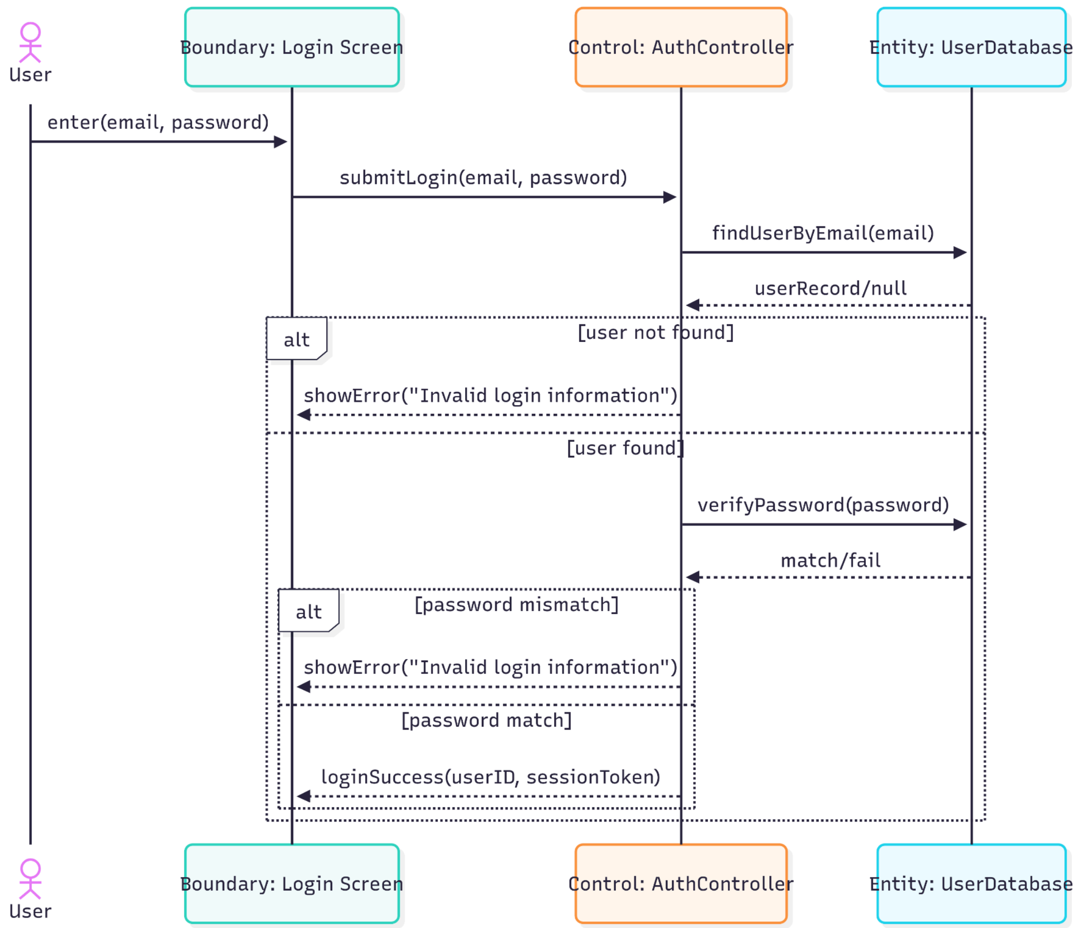
5. Class Diagram



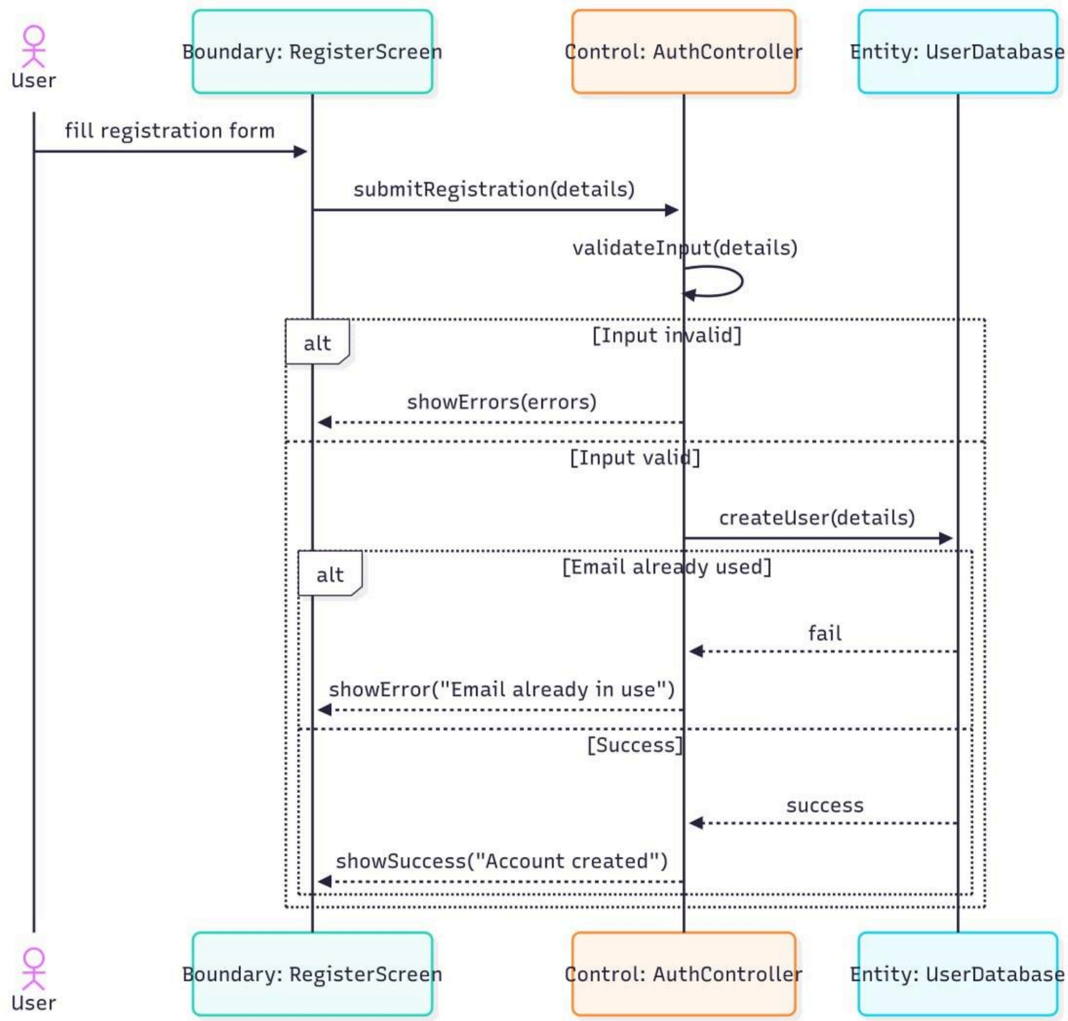
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.

6. Sequence Diagram

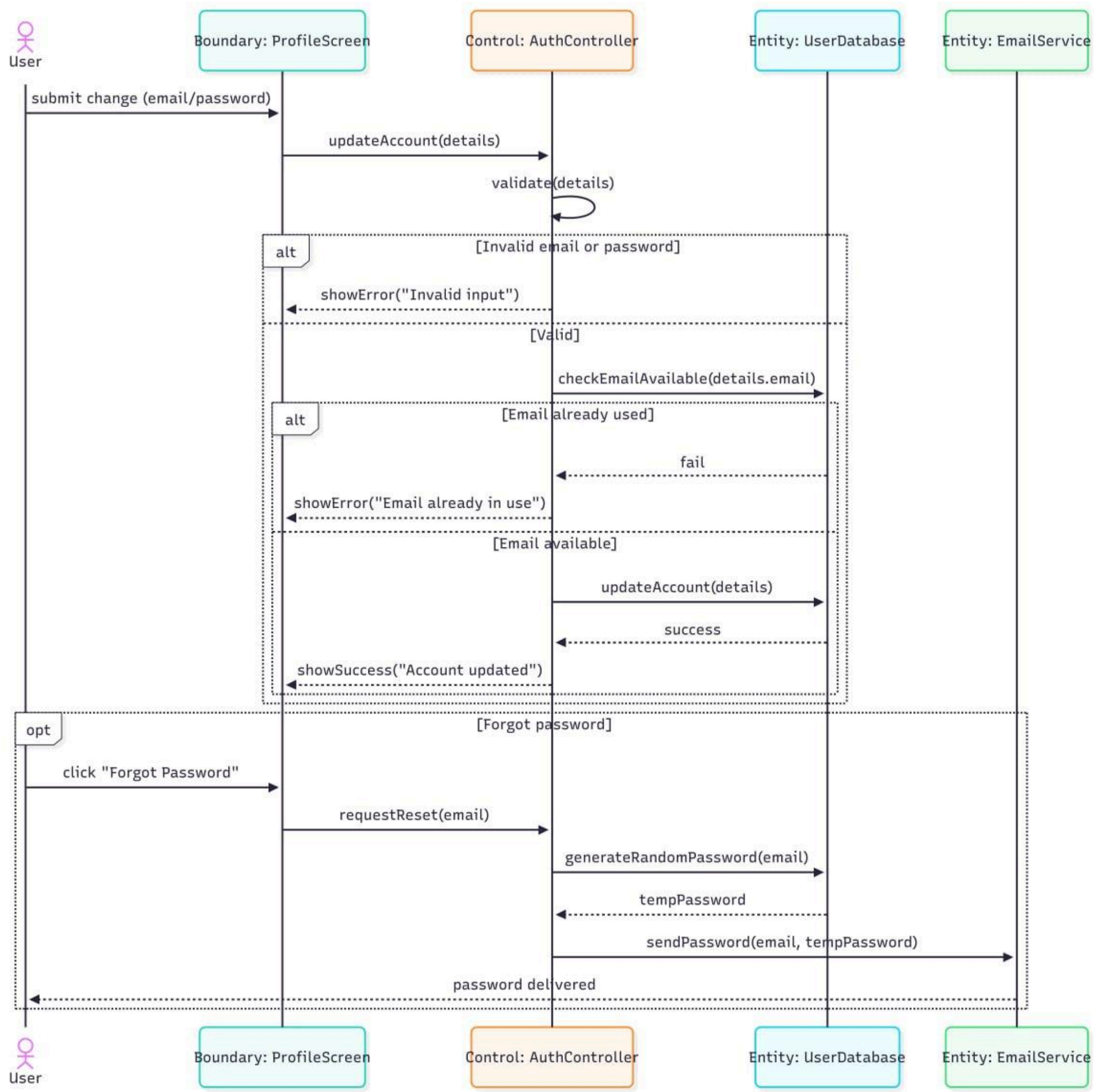
UC1 - Login



UC2 - Register

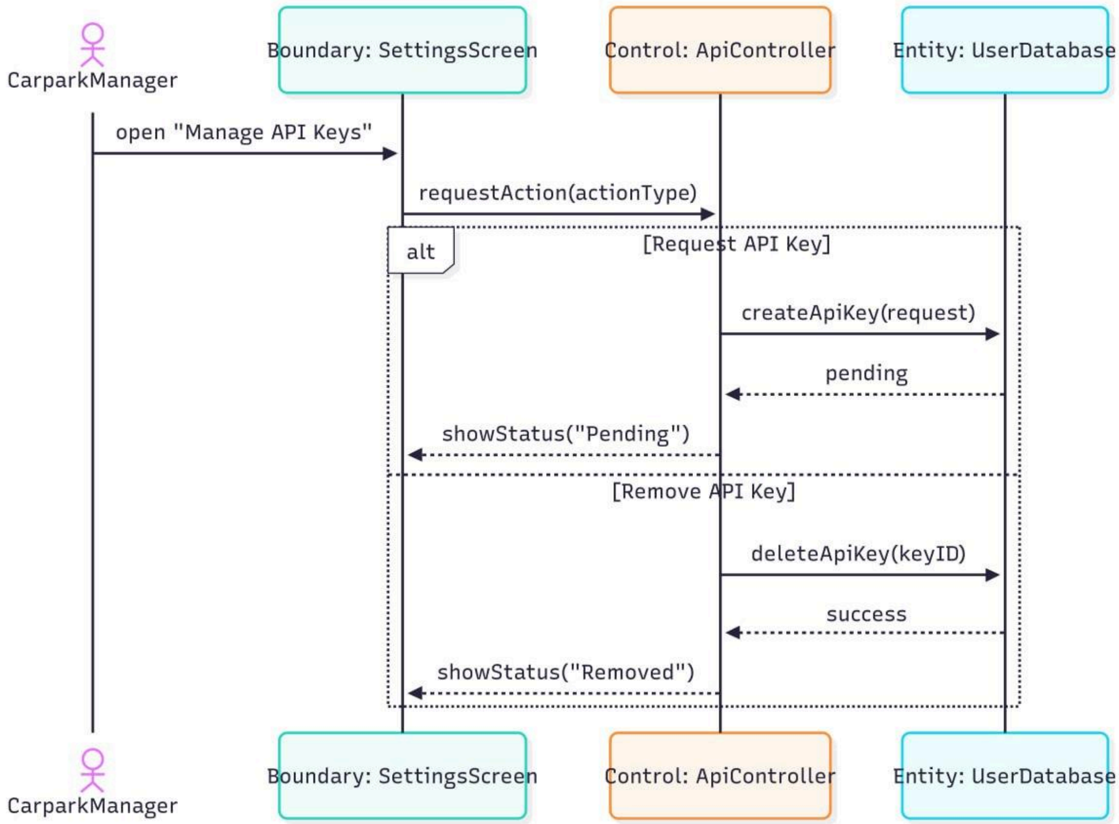


UC3 - Change Account Detail/ Forget Password

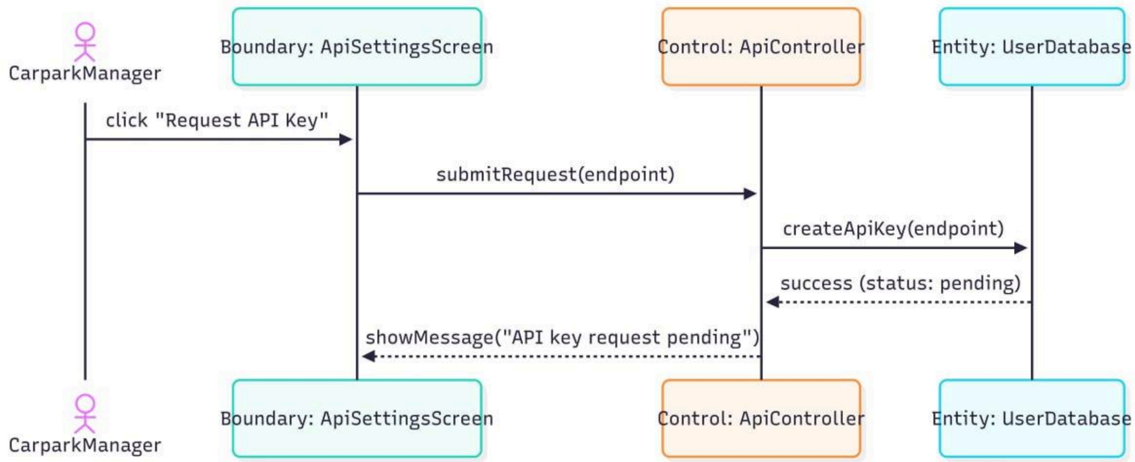


UC4 - Manage Carpark API Key

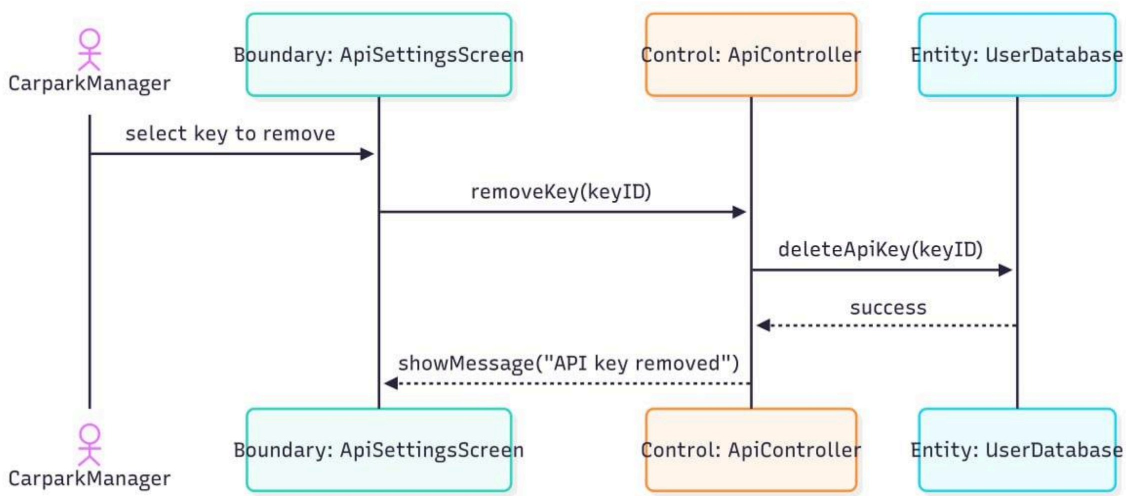
-



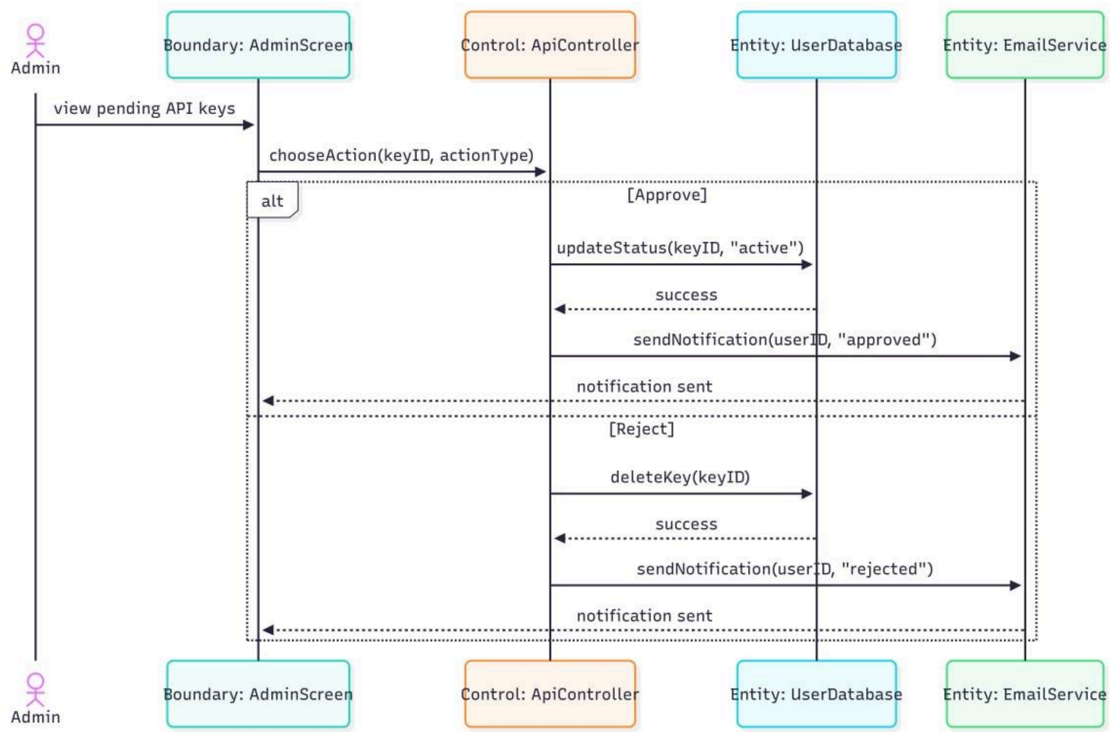
UC5 - Request Carpark API Key



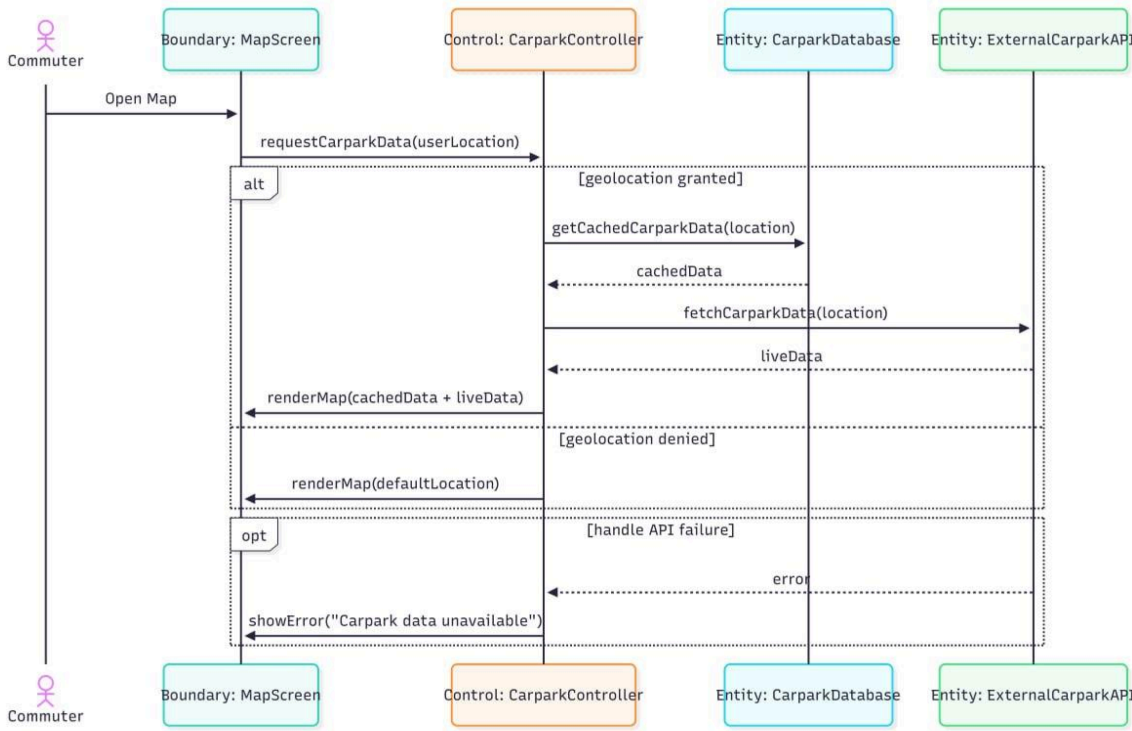
UC6 - Remove Carpark API Key



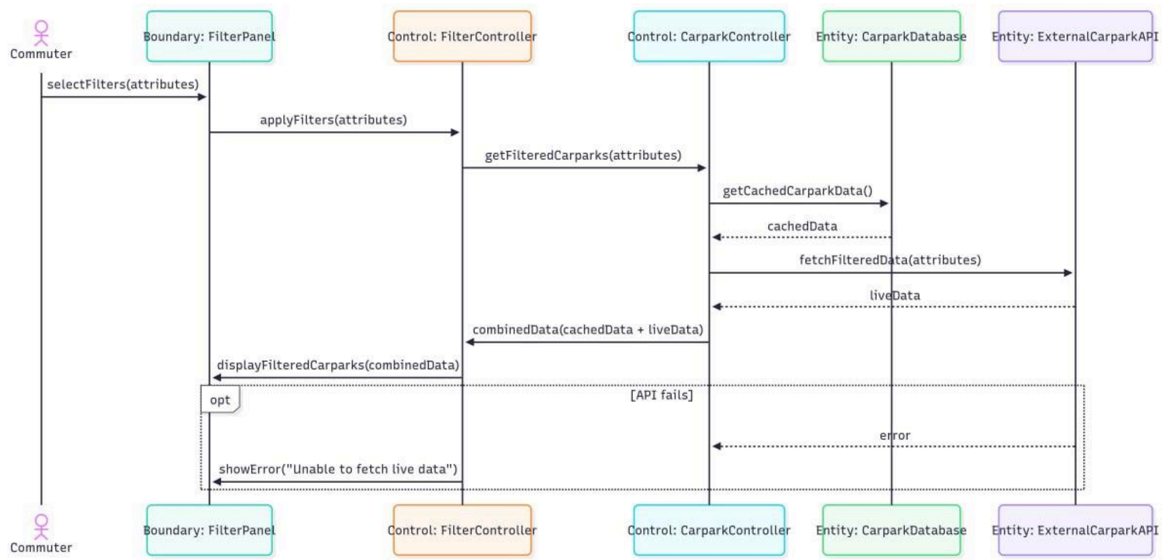
UC7, UC8, UC9 - Admin Manage API Key Status



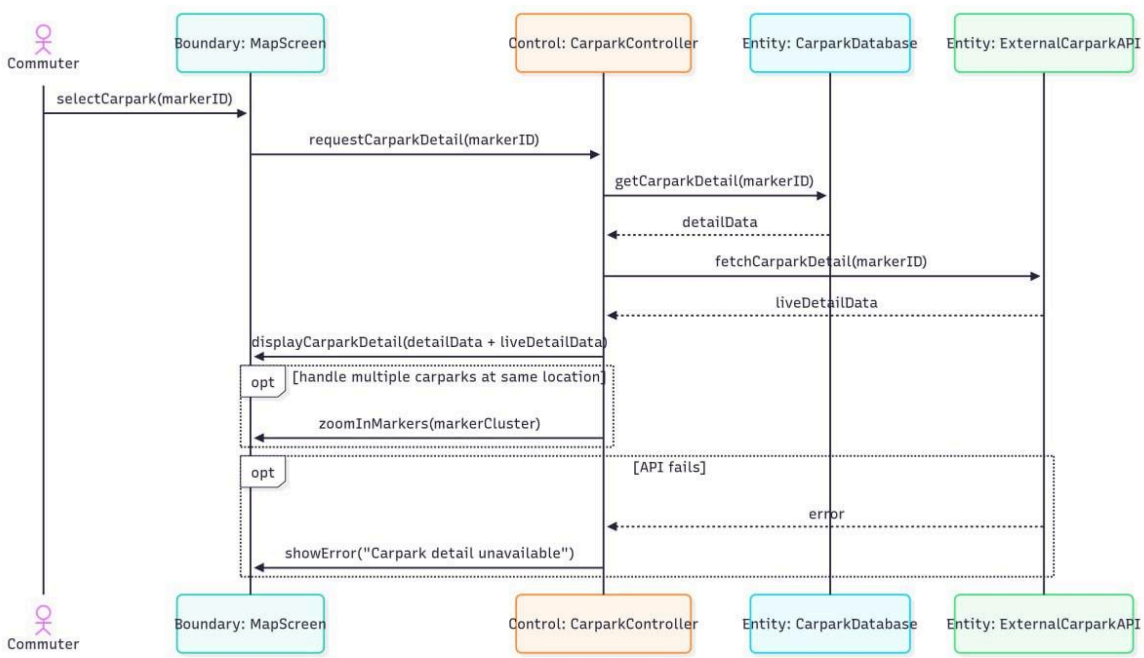
UC10 - View Carpark Map



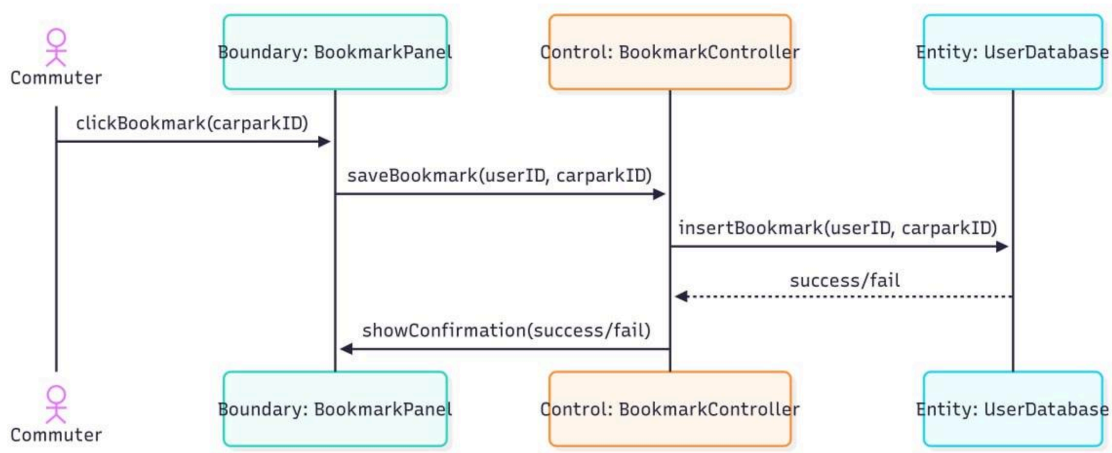
UC11 - Filter Carpark Map



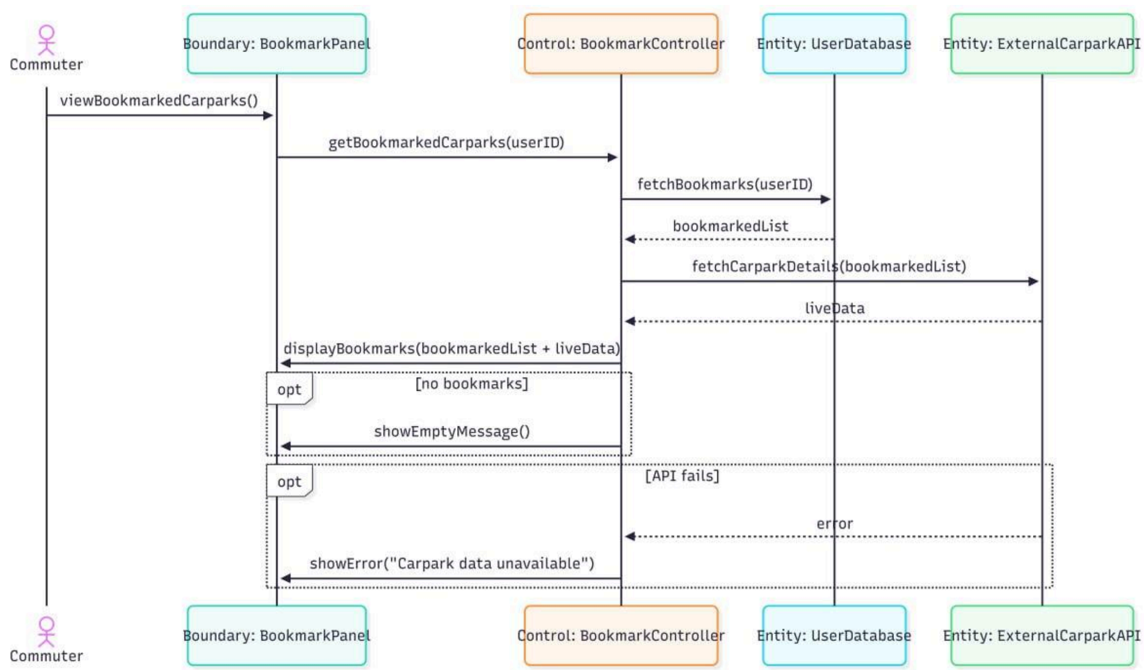
UC12 - View Carpark Details



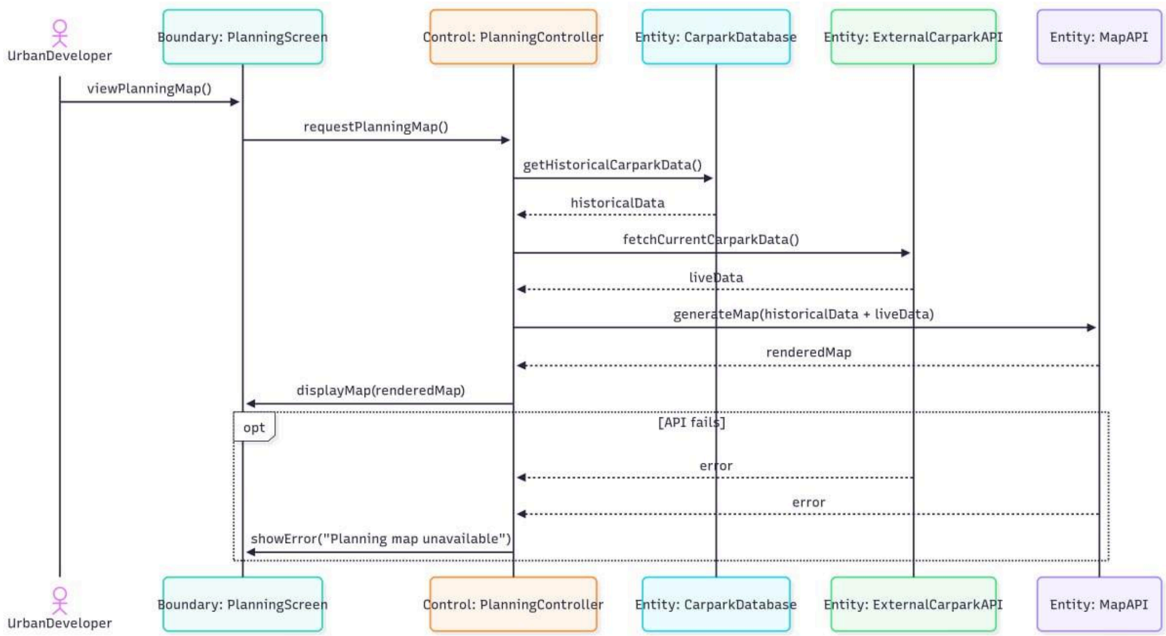
UC13 - Bookmark Carpark



UC14 - View Bookmark Carpark

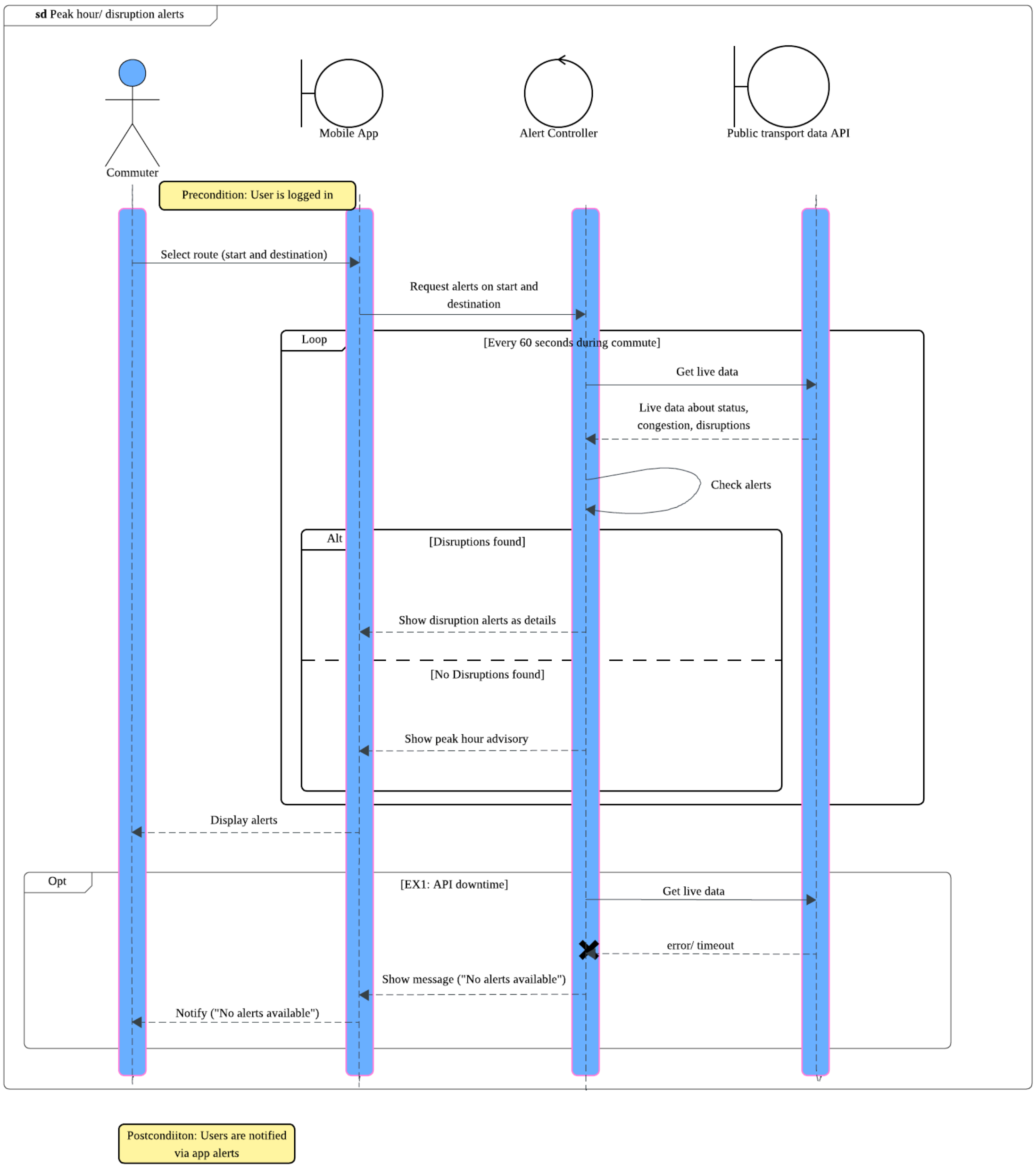


UC15 - View Carpark Planning Map

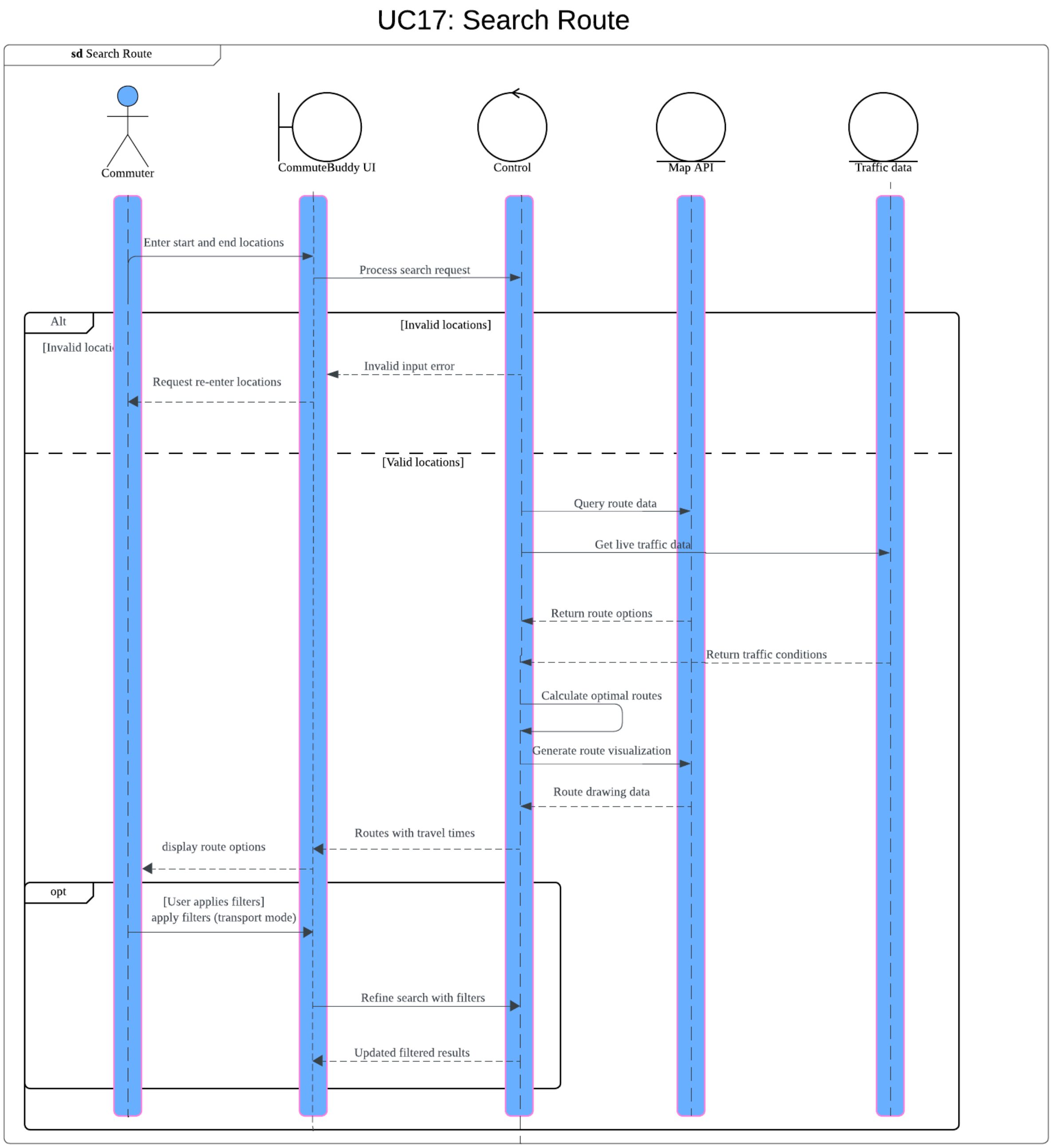


UC16 - Peak Hour/Disruption Alerts

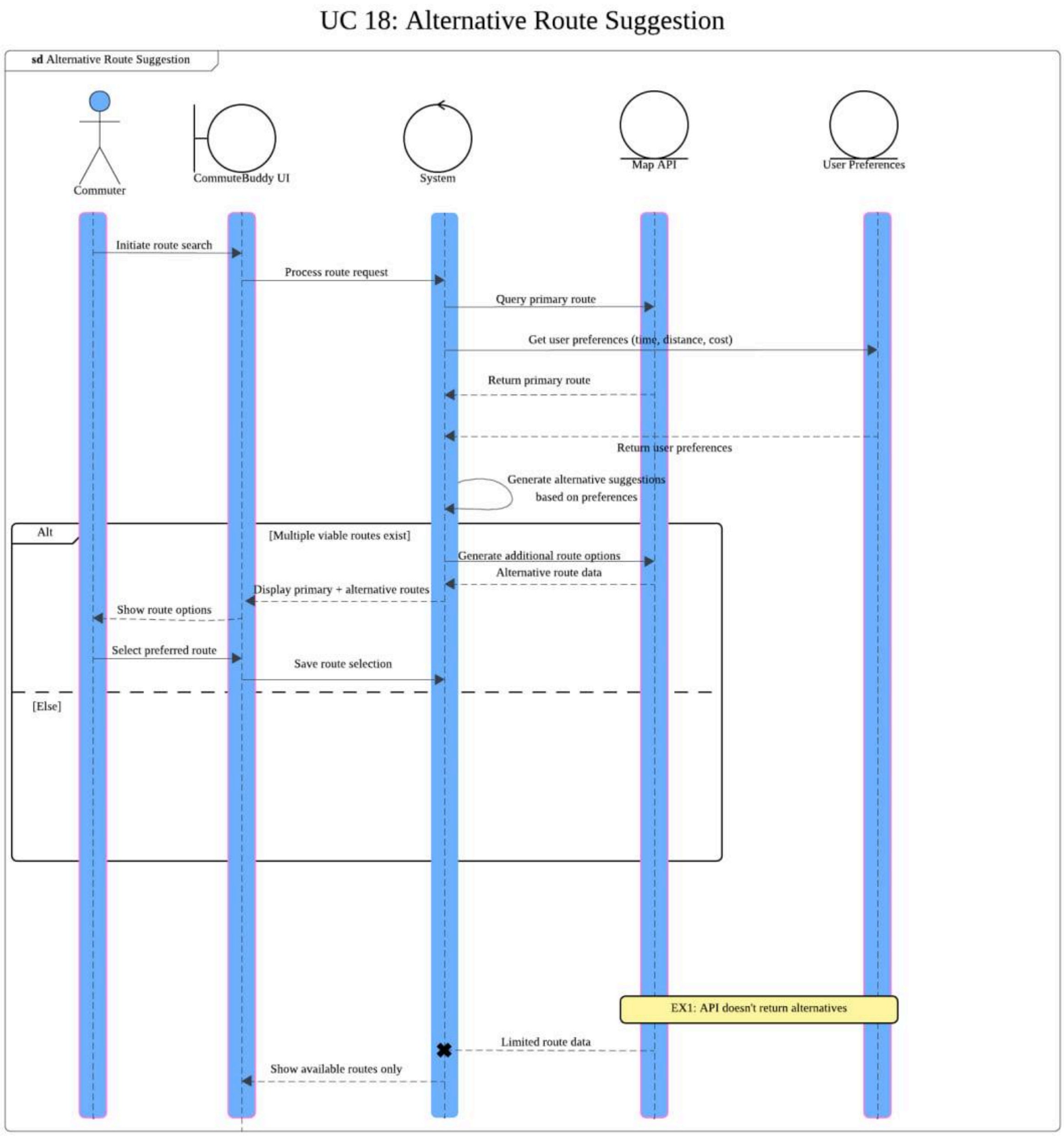
UC16: Peak hour/ Disruption alrts



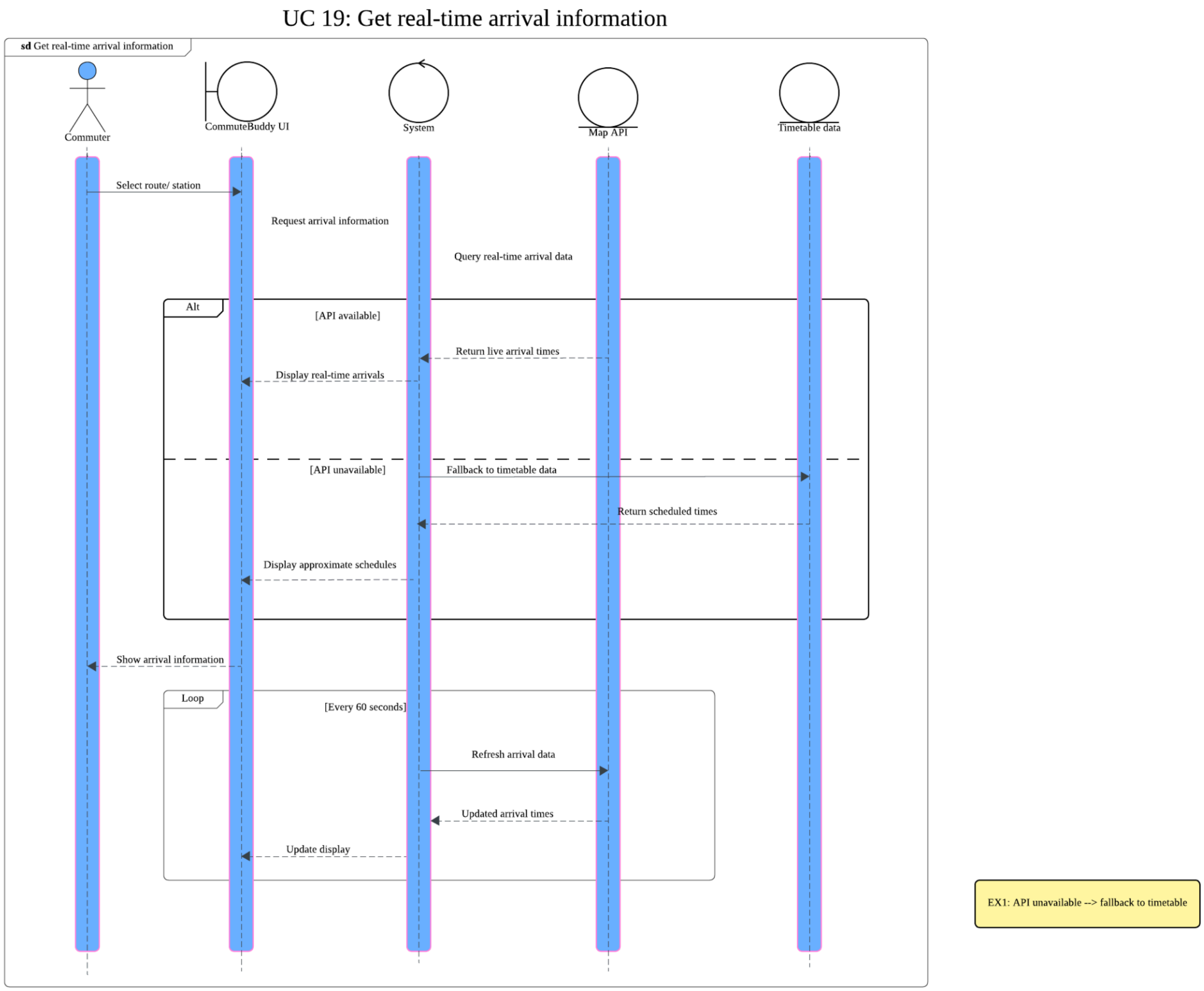
UC17 - Search Route



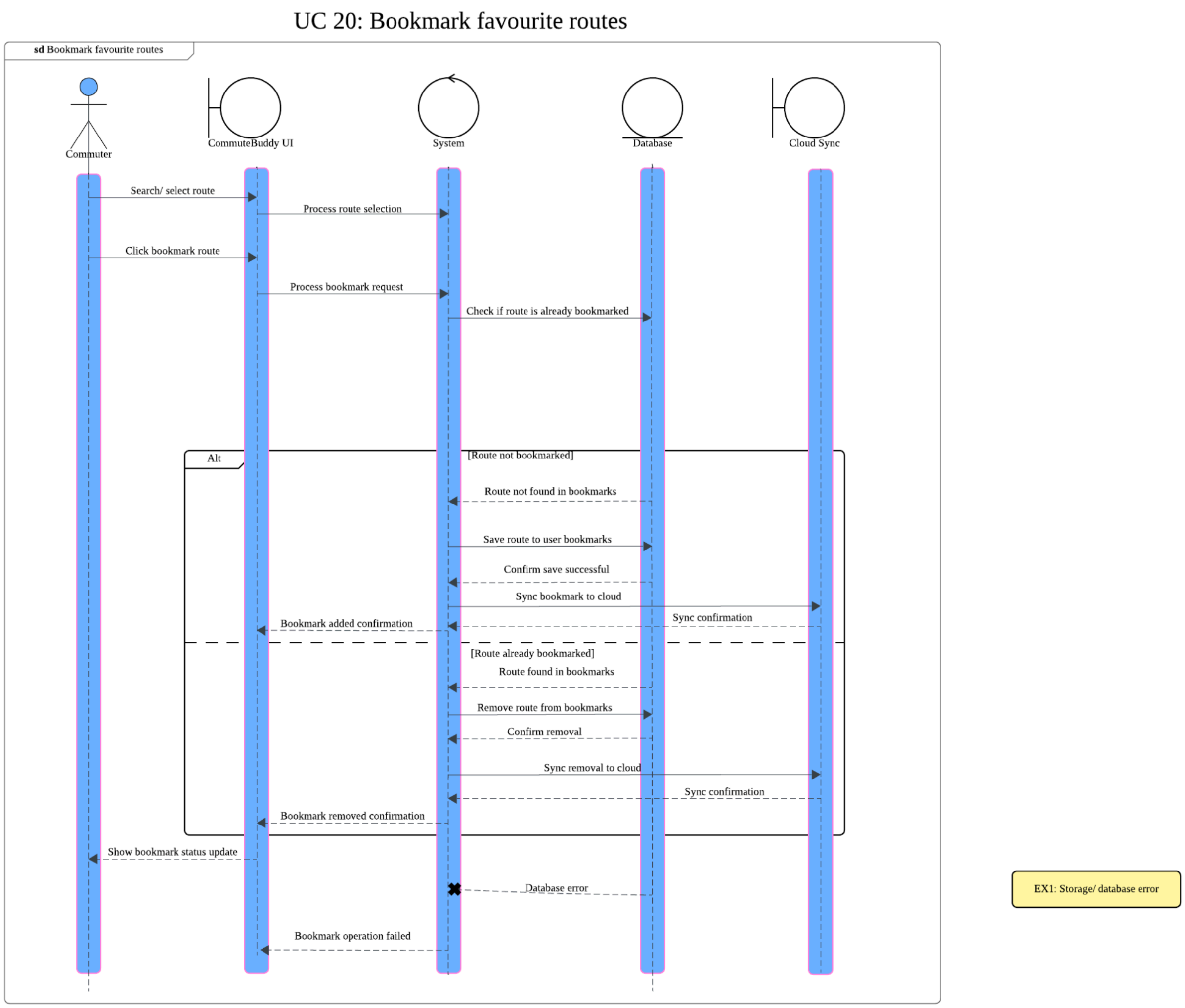
UC18 - Alternative route suggestion



UC19 - Get real-time arrival information

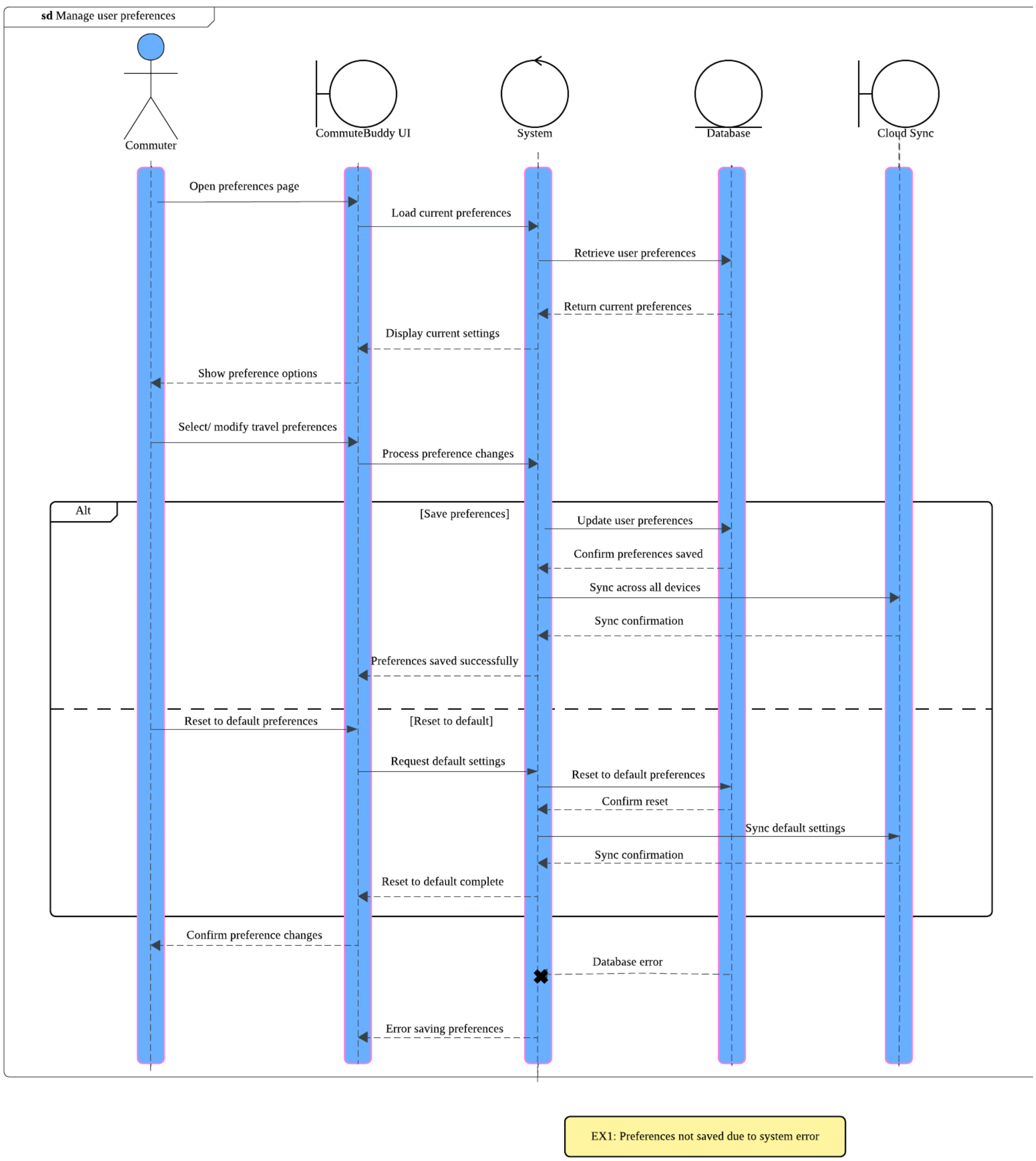


UC20 - Bookmark favourite routes



UC21 - Manage user preferences

UC 21: Manage user preferences

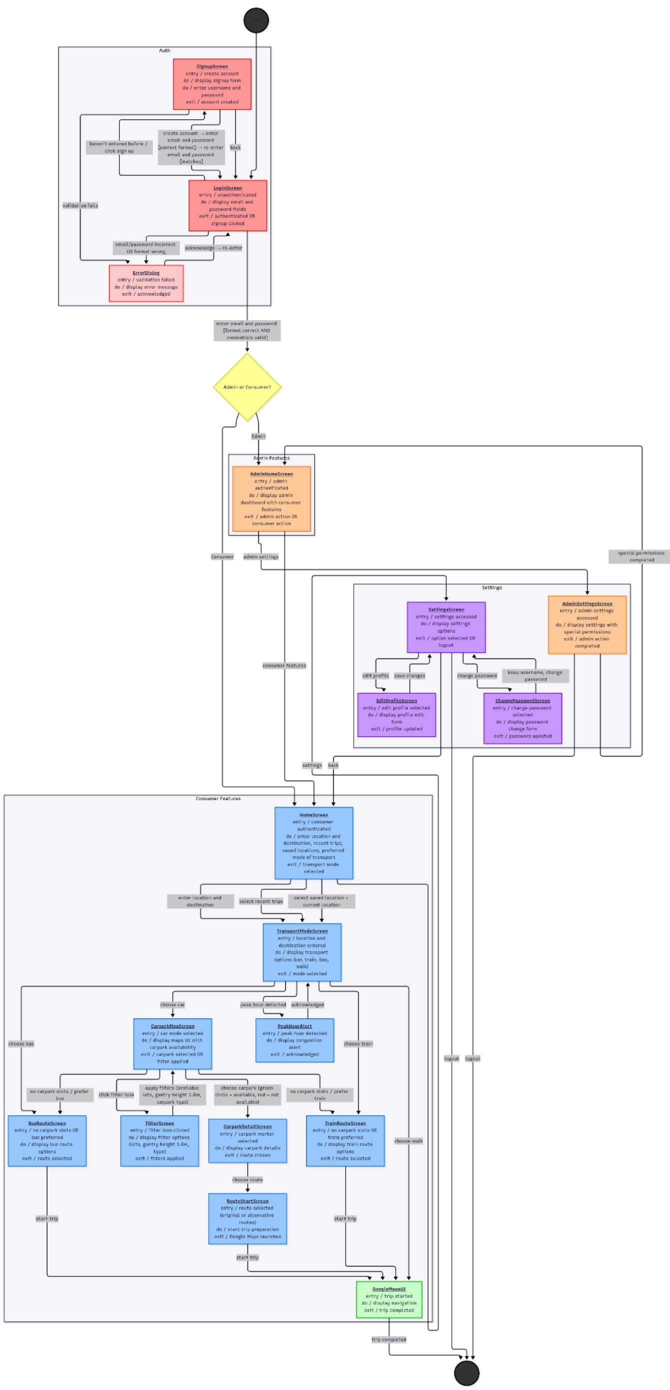


UC22 - View Historical data

UC 22: View historical data

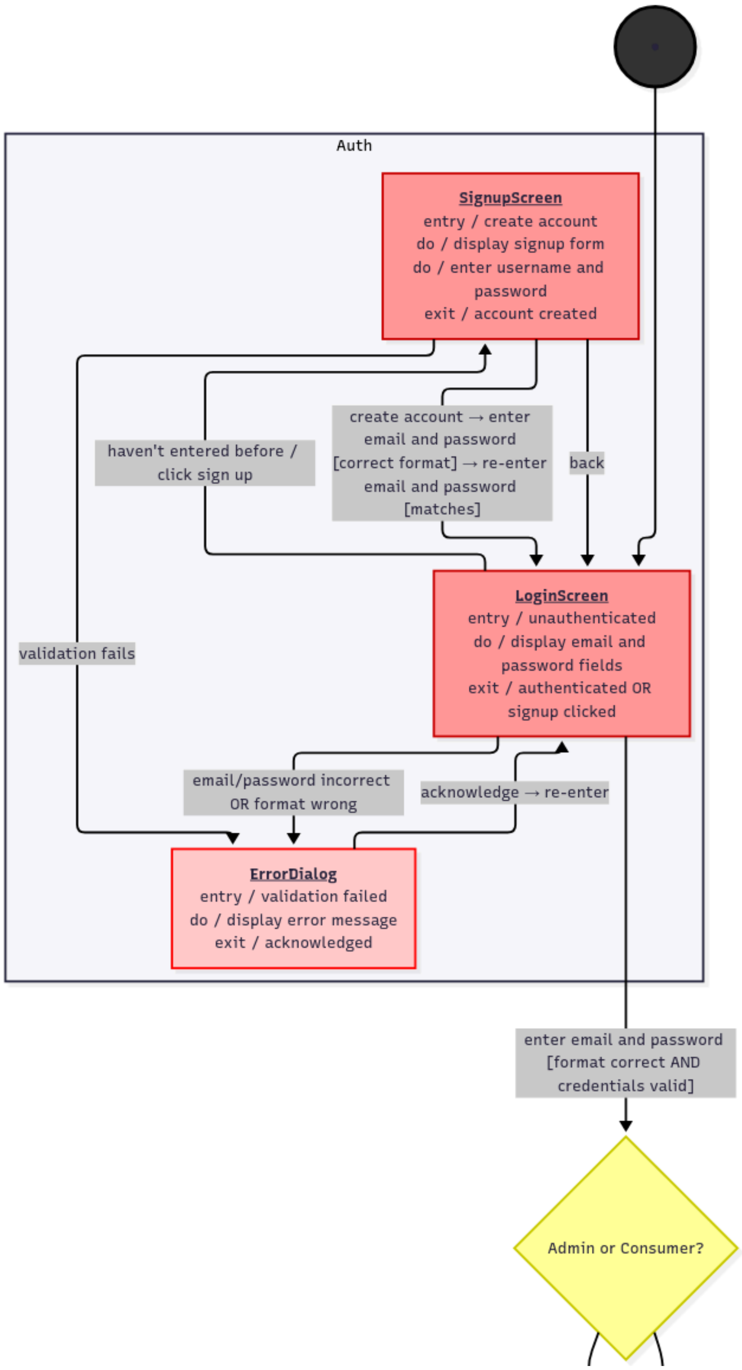
7. Dialog Map

7.1. Overview



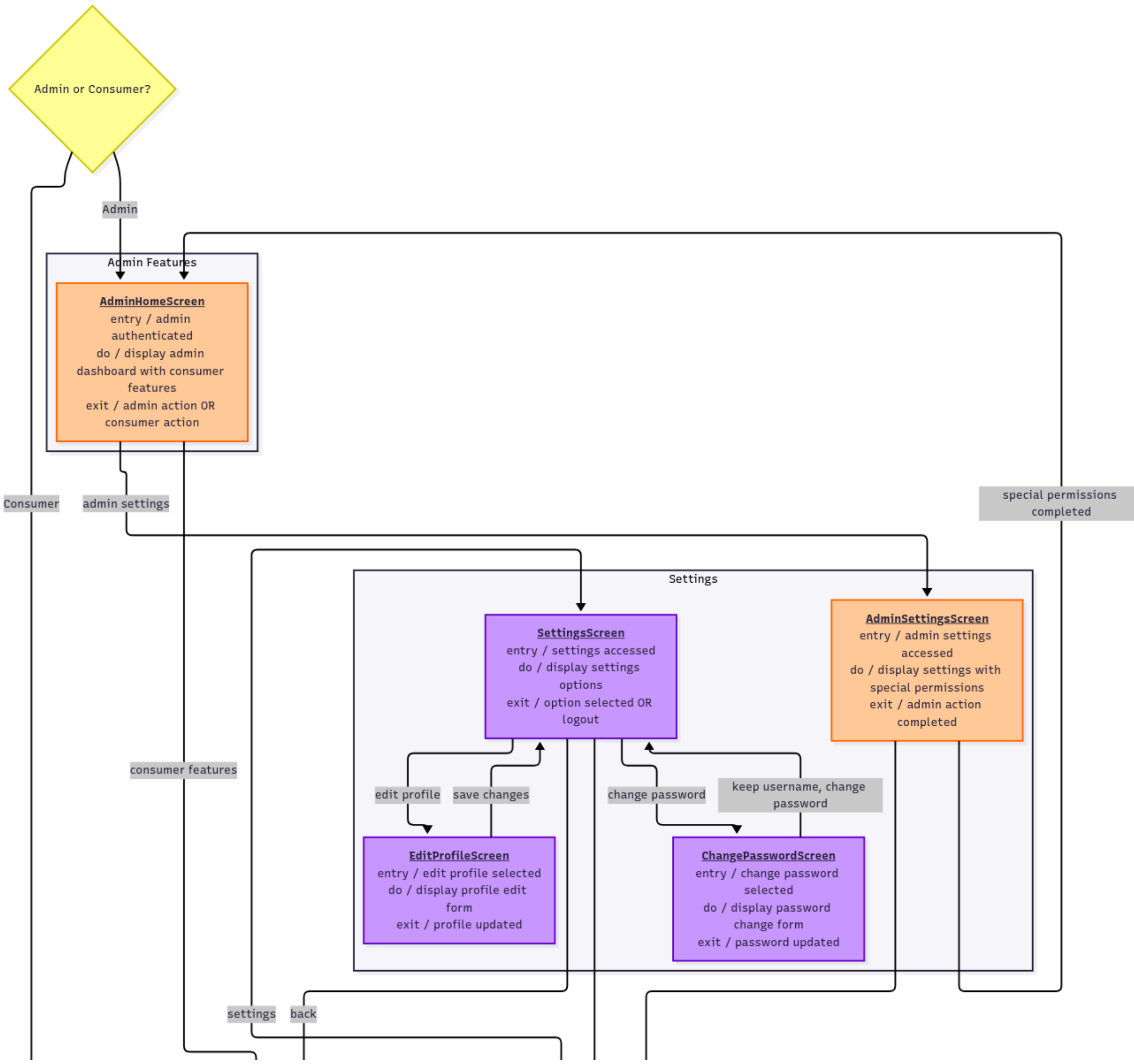
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.2. Authentication



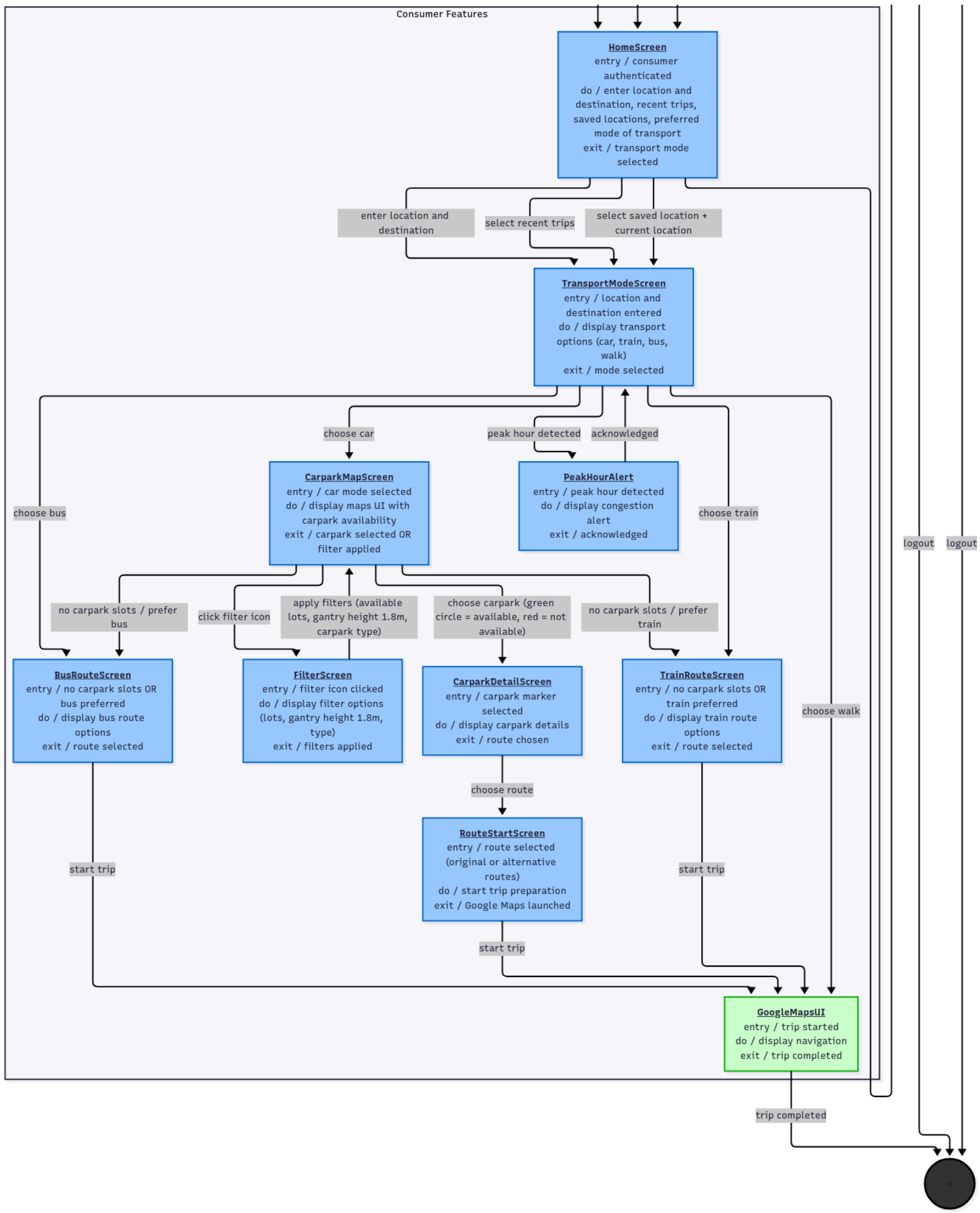
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.3. Consumer Settings and Admin Settings



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.4. Consumer Features



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

8. References

1. *Real-time carpark availability in Singapore*. CarparkSG. (n.d.). https://carparksg.com/
2. Google. (n.d.). Google maps. https://www.google.com/maps